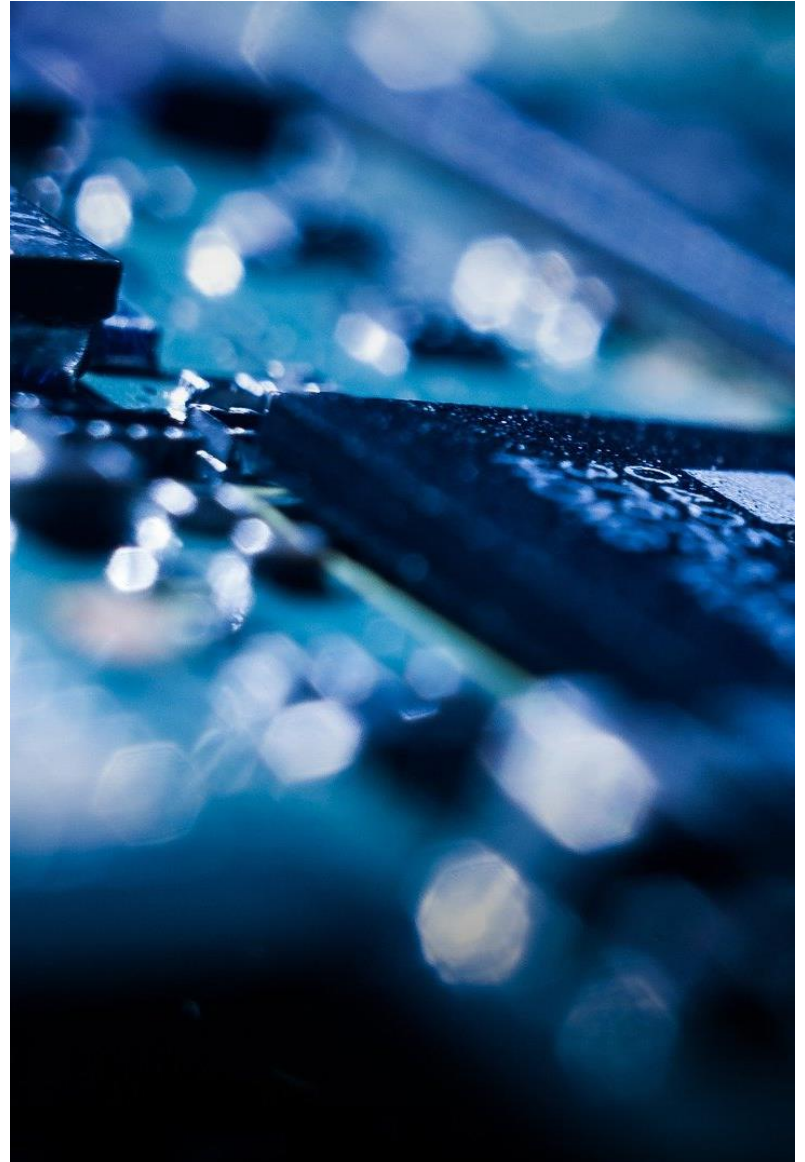


Programmierung eingebetteter Systeme

Einleitung und Grundlagen

Prof. Dr. Henning Trsek

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



Überblick – Grundlagen

- ▶ Einleitung
- ▶ Datenablage im Speicher
- ▶ Programmbeispiel
- ▶ Registermodell
- ▶ Daten bewegen
- ▶ Adressierungsarten
- ▶ Daten vergleichen und Sprungbefehle

Einsatzgebiete

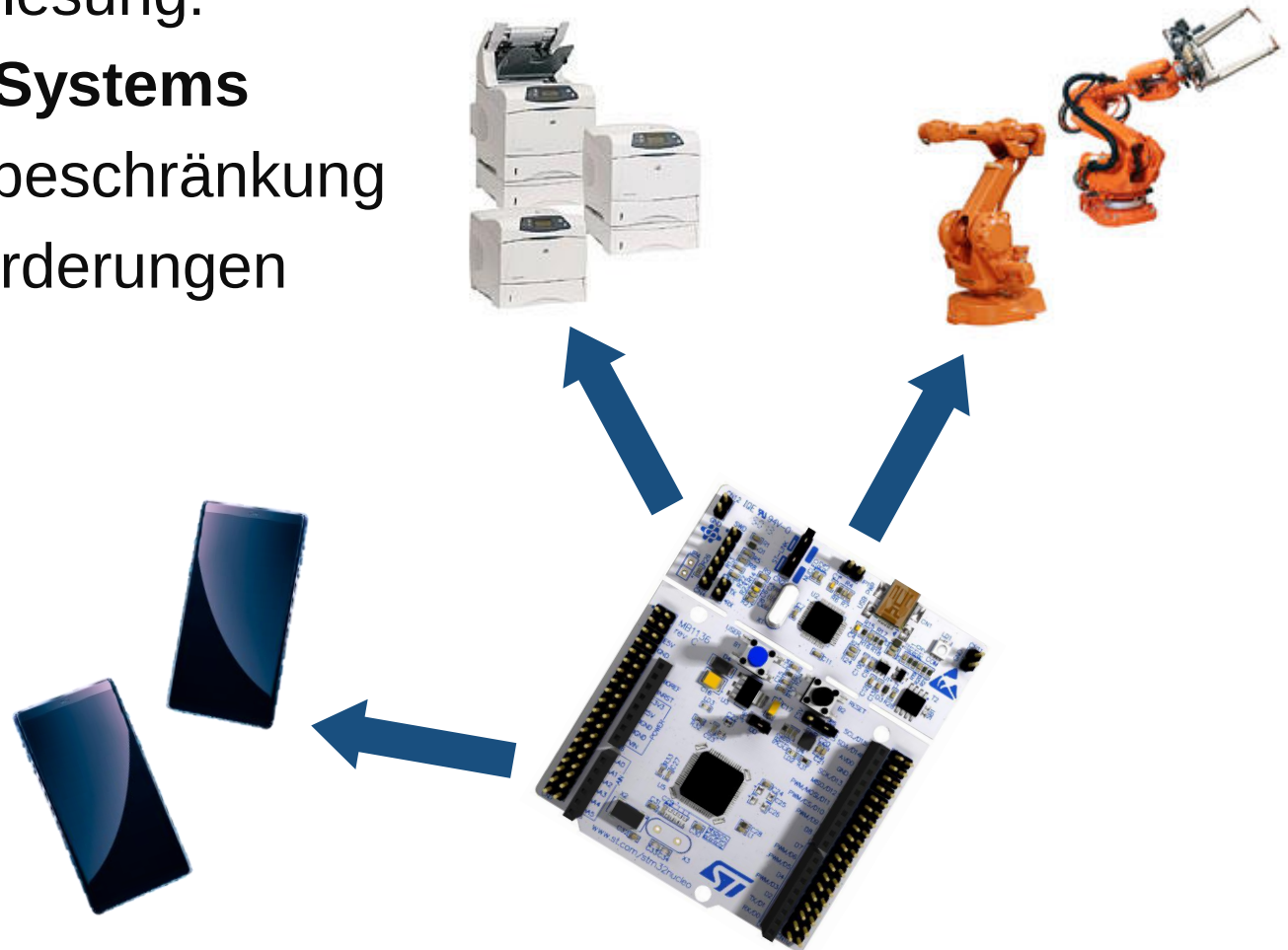
bisher:

**allgemeiner Rechner-
Einsatz** im Büro und in der
Industrie

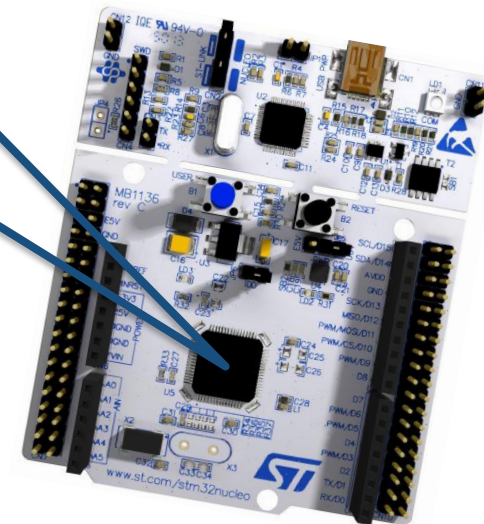
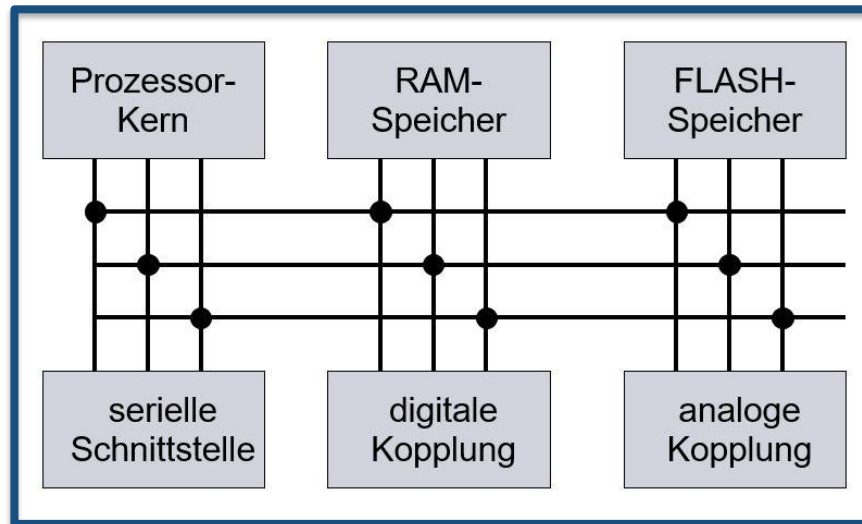


Einsatzgebiete

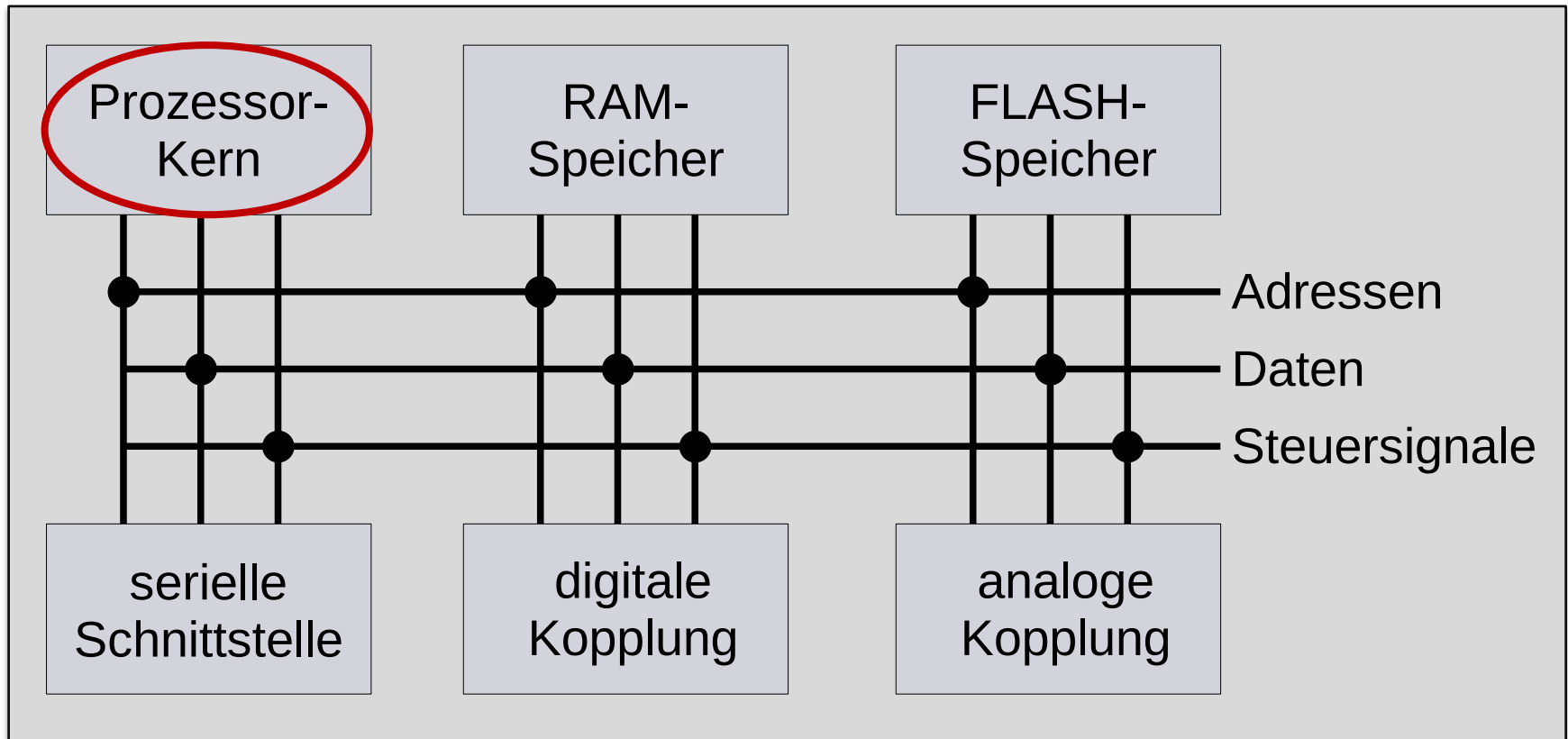
in dieser Vorlesung:
Embedded Systems
 Ressourcenbeschränkung
 Echtzeitanforderungen



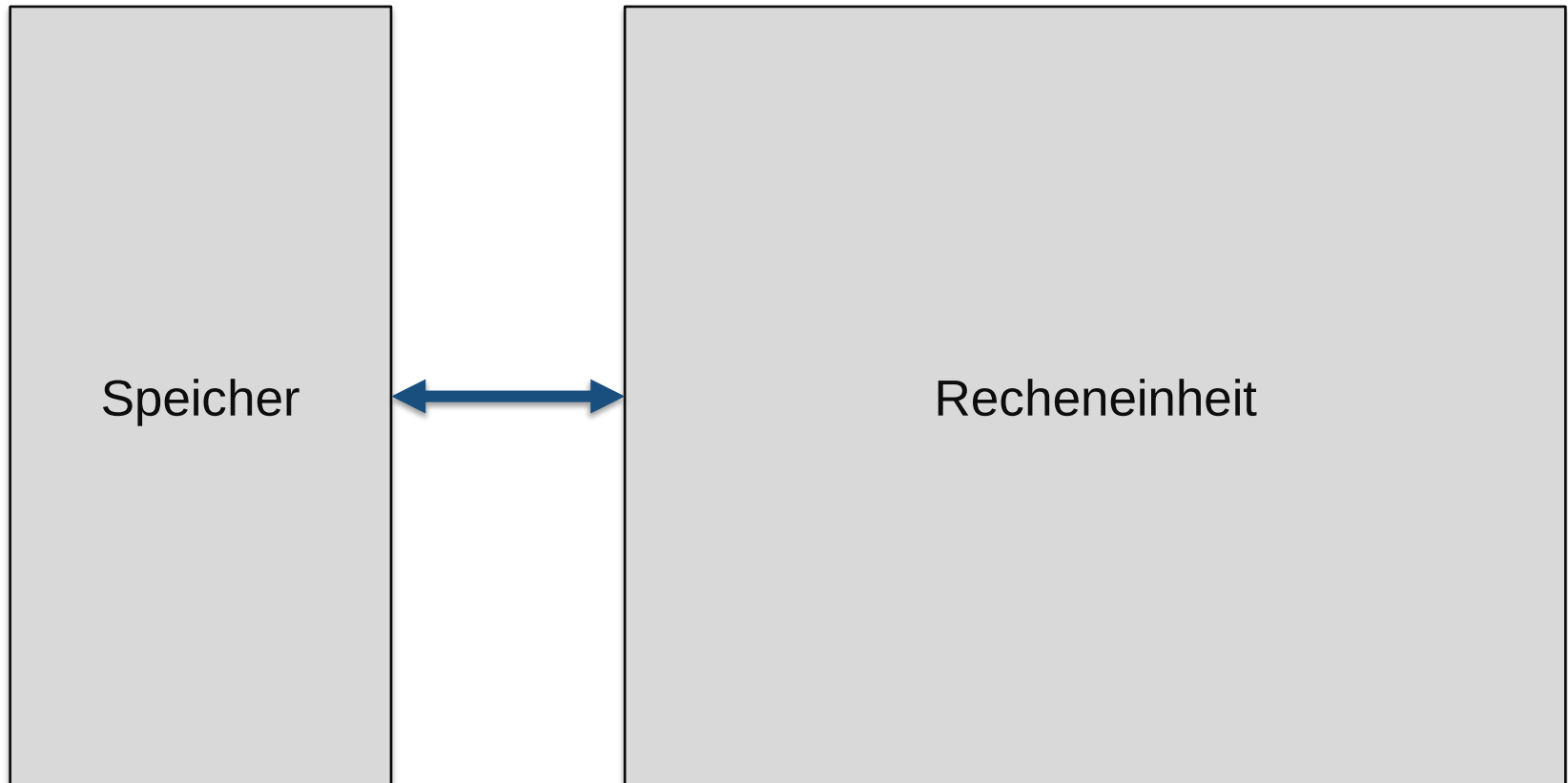
Aufbau Mikrocontroller



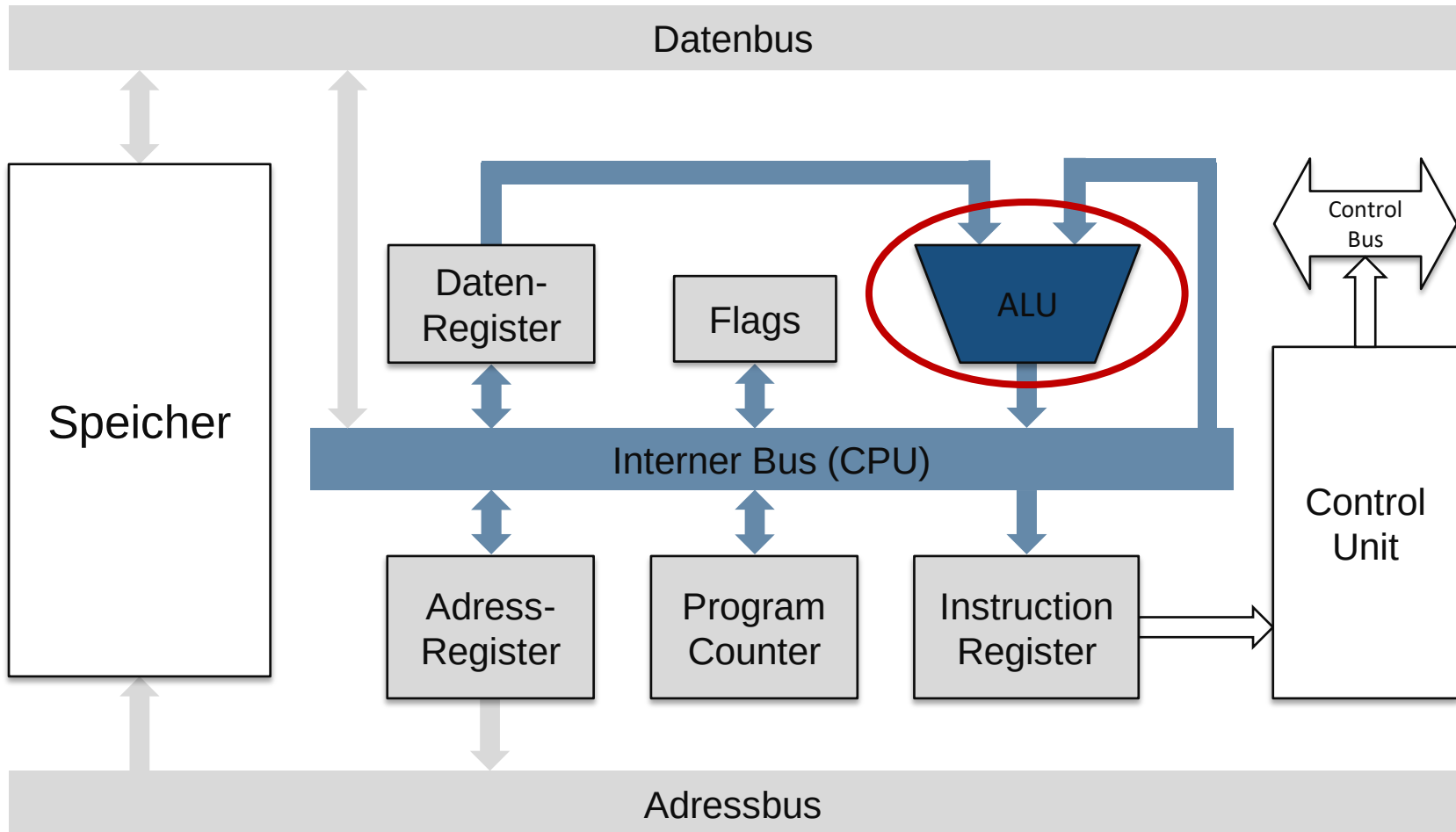
Blockschaltbild



Mikrocontroller (vereinfacht)



Mikrocontrollerstruktur (vereinfacht)



Aufbau Mikrocontroller

Bearbeiten von Daten

Rechnen mit Daten

positive und negative Zahlen

Addition, Subtraktion

Multiplikation, Division

Bewegen von Daten

Vergleichen von Daten

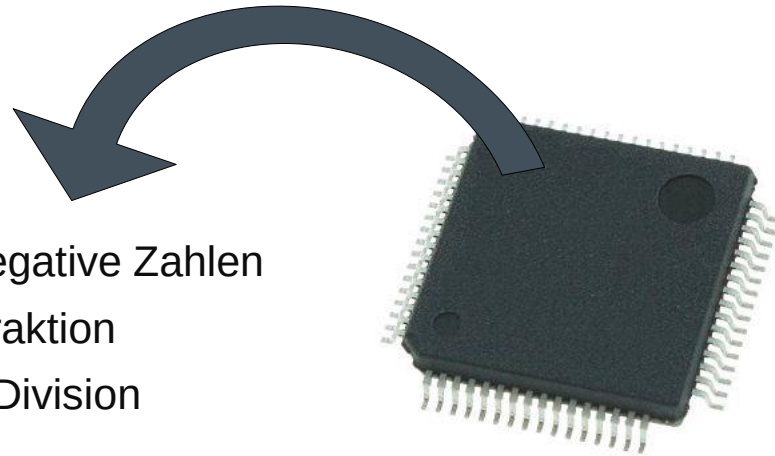
Logische Verknüpfung von Daten

Einzelbit-Verarbeitung

Programmflusskontrolle

Unterprogramme

Ausnahmeverarbeitung



STM32L476RG MCU

Datenablage im Speicher

Datenablage im Speicher



String von Einzelbits:

100111011100000001010011010000011110011...

Datenablage im Speicher



String von Einzelbits:

01010100 01100101 01100011 01101000
 01101110 01101001 01110011 01100011
 01101000 01100101 00100000 01001000
 01101111 01100011 01101000 01110011
 . . .

**Technische
Hochschule
Ostwestfalen-
Lippe...**

Datenablage im Speicher



Speicher (8 Bit Breite)

11010001
10101001
01010010
10010101
11110100
10000101

Byte-Adressierung

0x00000000	
0x00000001	
⋮	
⋮	
0xFFFFFFFFE	
0xFFFFFFFFF	

Datenablage im Speicher



Speicher (16 Bit Breite)

Datenbreite

11010001
10101001
01010010
10010101
11110100
10000101

Nibble (4 Bit)

Datenablage im Speicher



Speicher (8 Bit Breite)

Datenbreite

11010001
10101001
01010010
10010101
11110100
10000101

Byte (8 Bit)

Datenablage im Speicher



Speicher (8 Bit Breite)

Datenbreite

11010001
10101001
01010010
10010101
11110100
10000101

Word (16 Bit)

Datenablage im Speicher



Speicher (8 Bit Breite)

Datenbreite

11010001
10101001
01010010
10010101
11110100
10000101

Long (32 Bit)

Datenablage im Speicher



Speicher (8 Bit Breite)

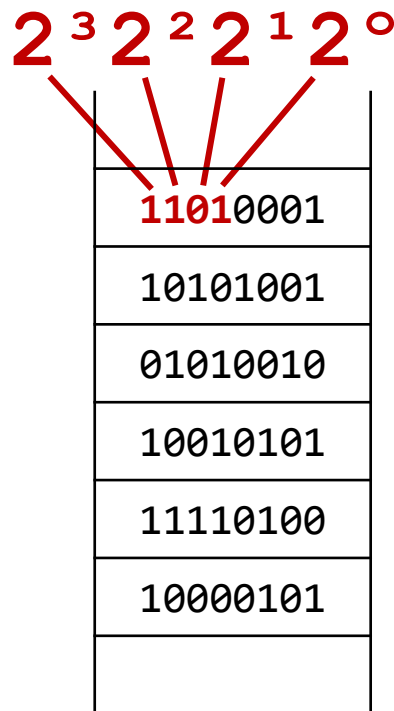
11010001
10101001
01010010
10010101
11110100
10000101

Byte-Adressierung

0xD1
0xA9
0x52
0x95
0xF4
0x85



Datenablage im Speicher



Byte-Adressierung

0xD1
0xA9
0x52
0x95
0xF4
0x85

Datenablage im Speicher



Speicher (16 Bit Breite)

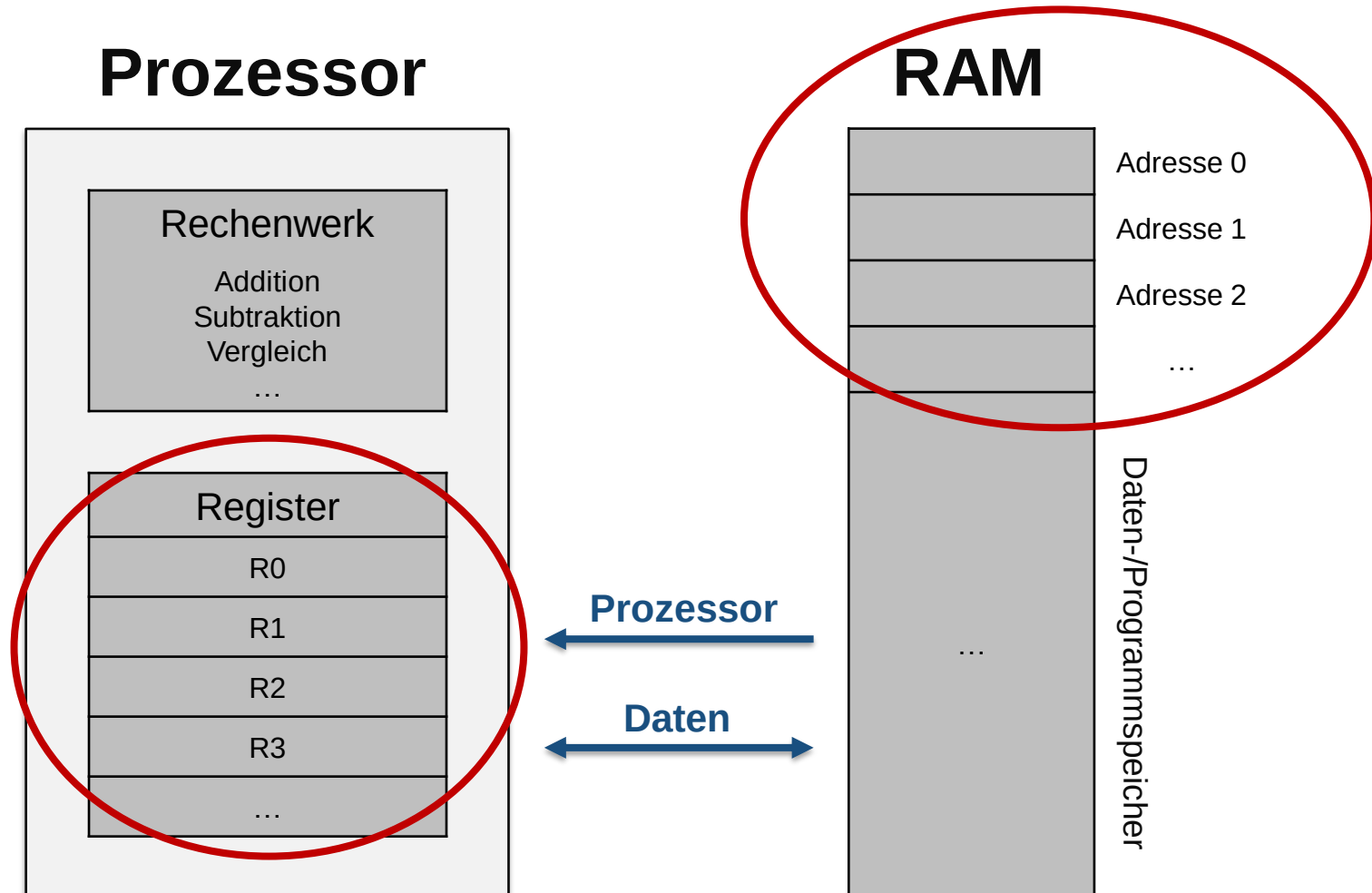
11010001
10101001
01010010
10010101
11110100
10000101

Byte-Adressierung

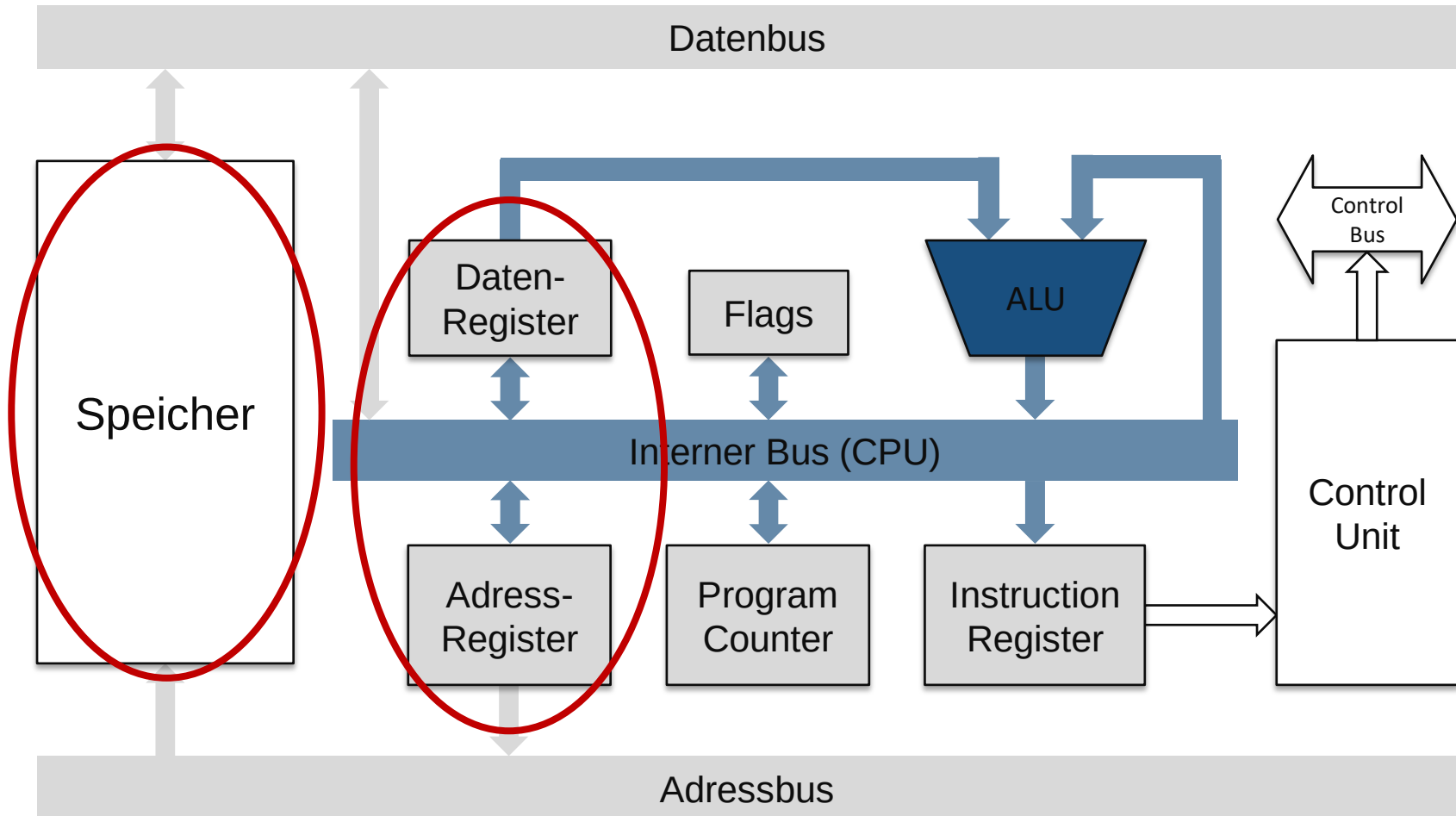
0xD1
0xA9
0x52
0x95
0xF4
0x85



Register – Speicher

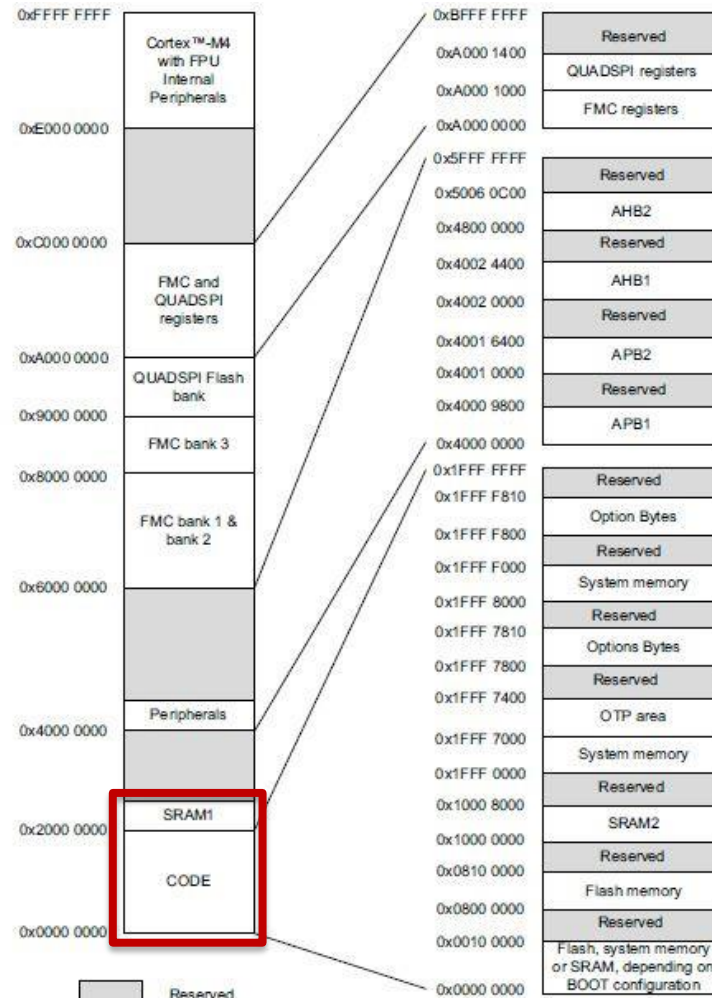


Register – Speicher



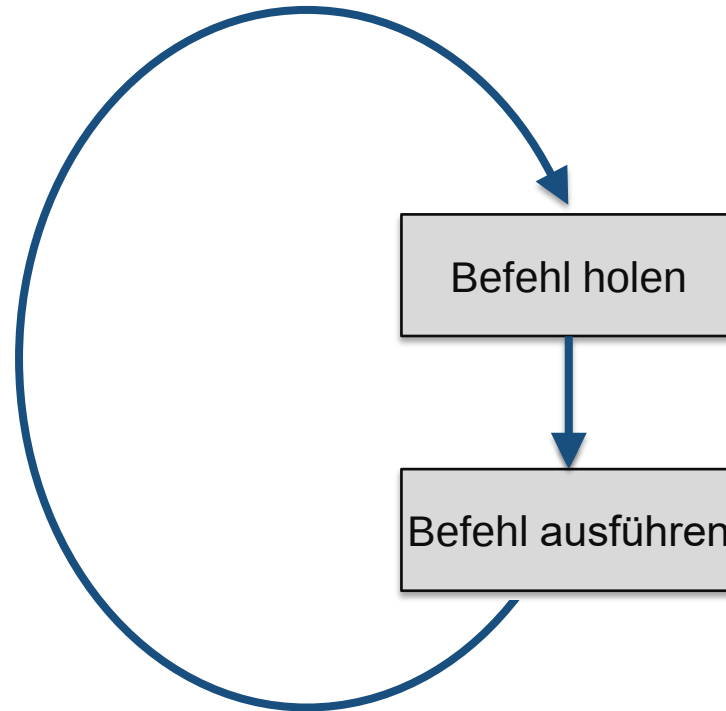
Auf welchen Adressen findet der Prozessor den Speicher?

Speicheraufteilung (Memory Map)



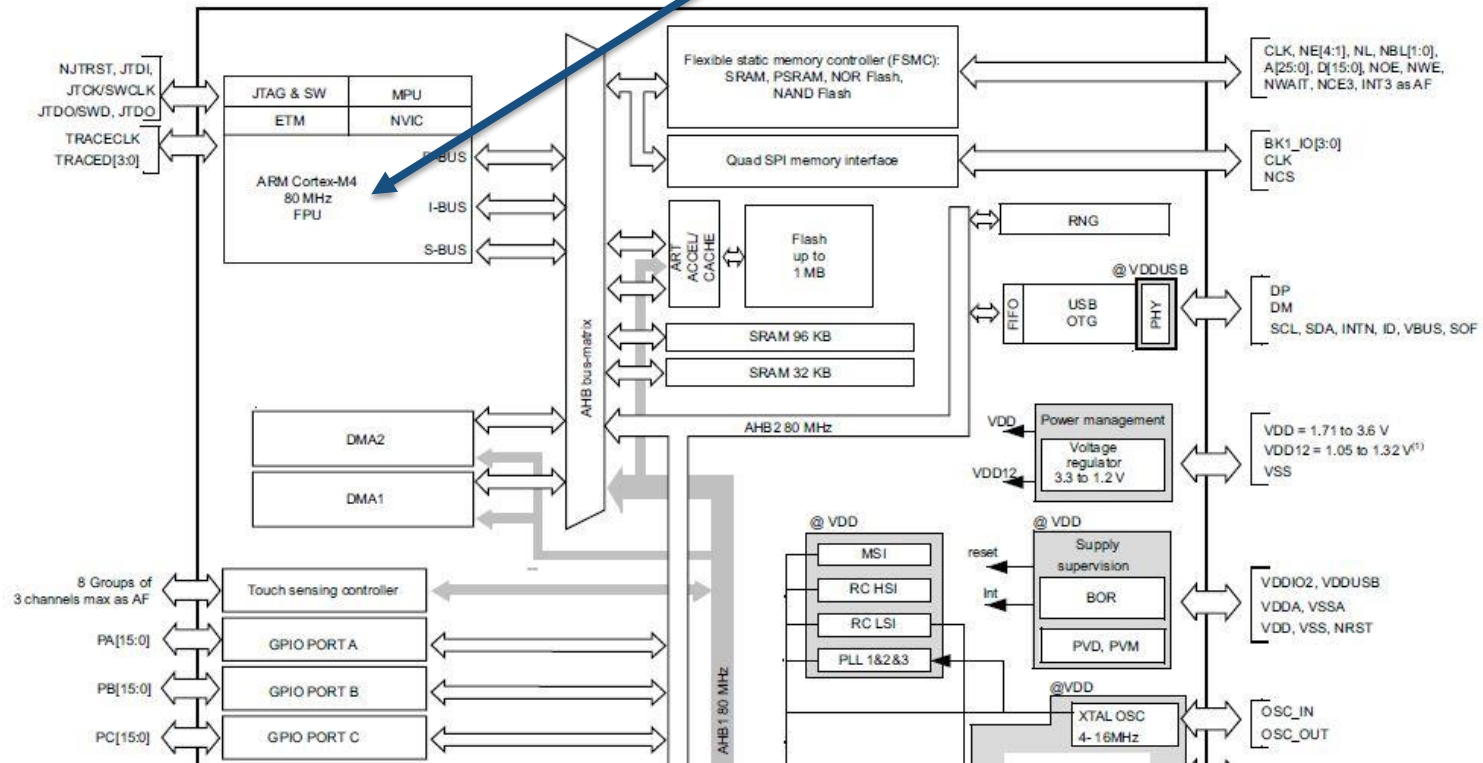
Programm-Beispiel

Befehlsschleife

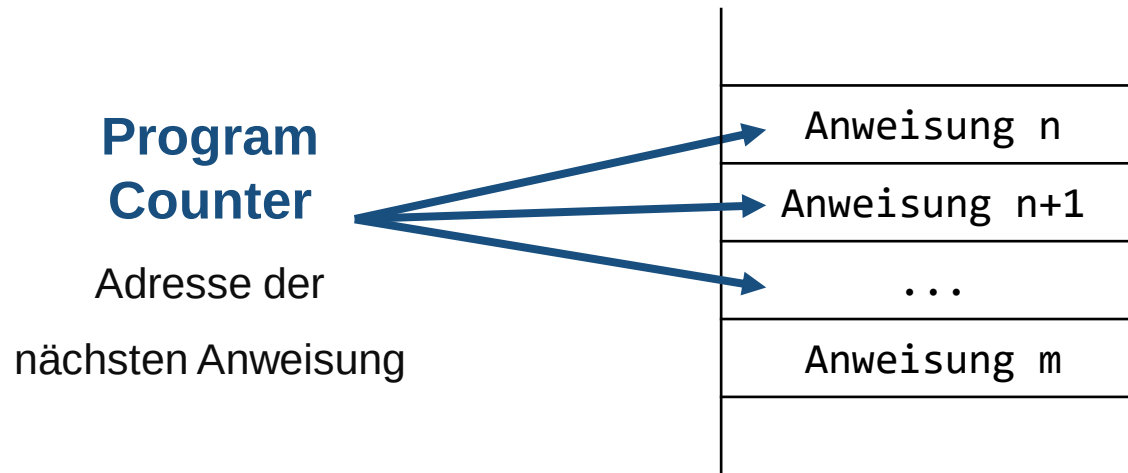


CPU-Kern

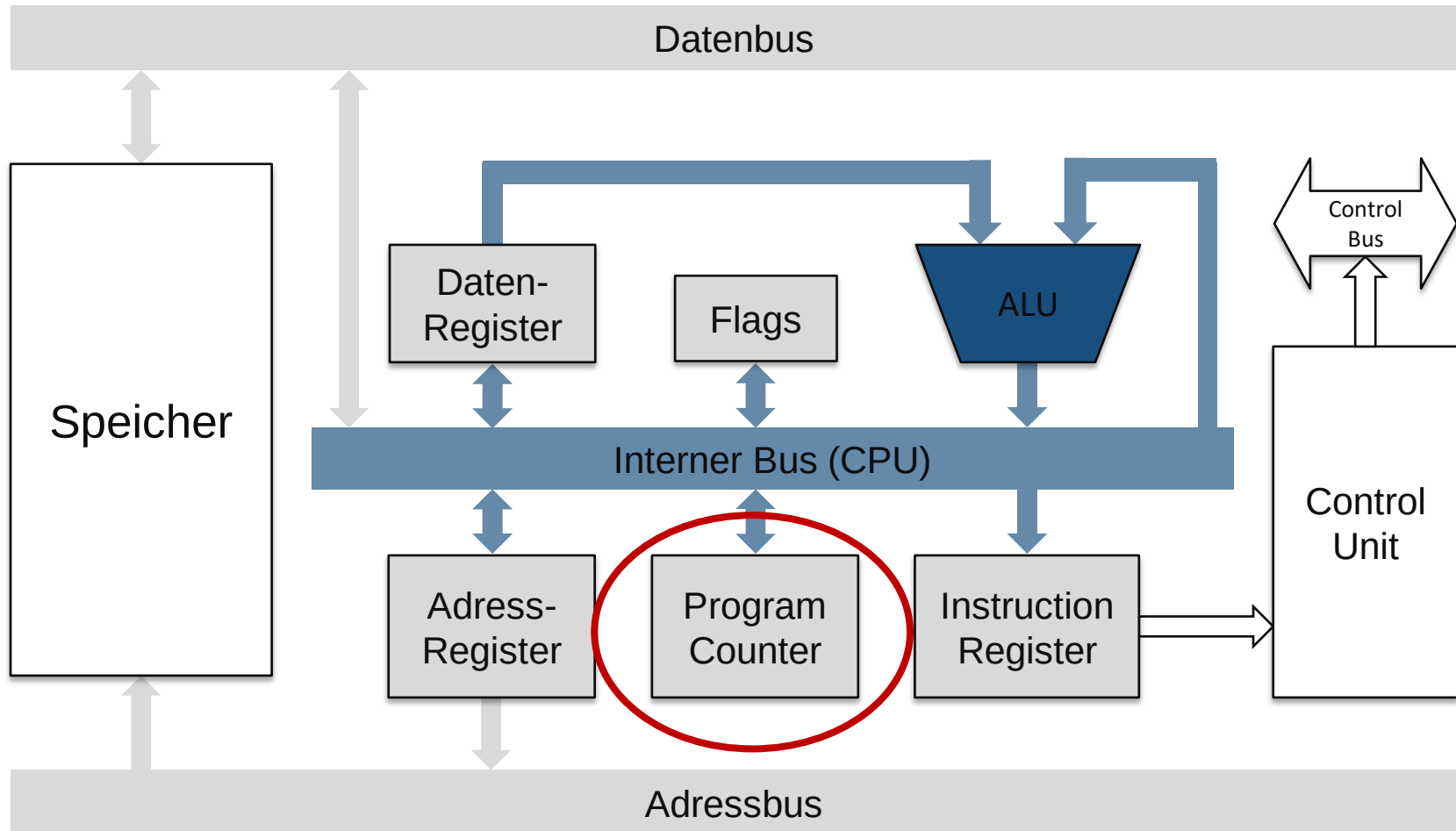
Prozessor



Program Counter



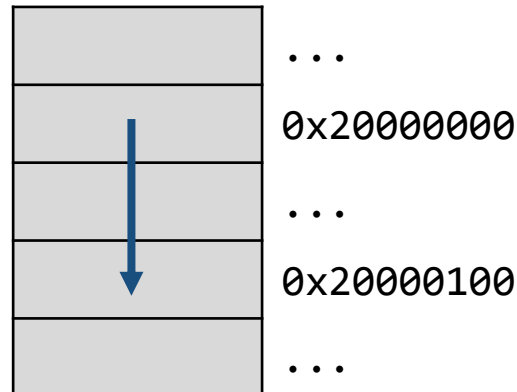
Program Counter



Anweisungen

Bewege (kopiere) ein Word Daten von der Speicheradresse 0x20000000 an die Speicheradresse 0x20000100.

hexadezimal



Anweisungen

Bewege (kopiere) ein Word Daten von der Speicheradresse 0x20000000 an die Speicheradresse 0x20000100.

Anweisung im Speicher:

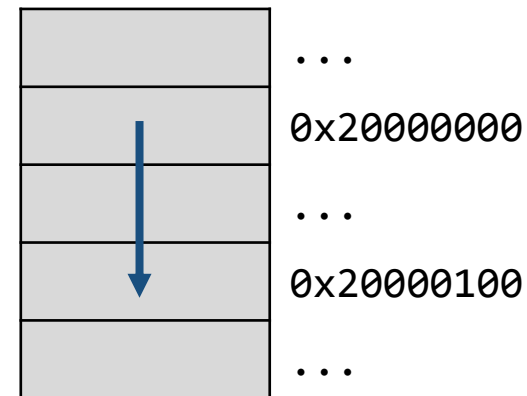
0001 1000 0000 1100 0011 0001 0101 0100 0011 0010 1111 0110

Hexadezimale Schreibweise:

18 0C 31 54 32 F6

Symbolische Schreibweise (Assembler):

```
ldr r0, =0x20000000
ldr r1, =0x20000100
ldr r2, [r0]
str r2, [r1]
```



Programmbeispiel

```

.syntax unified
.global asm2a
.text
asm2a:  adr  r0, tab1
        ldr  r1, =0x20000000
loop:   ldr  r2, [r0], #4
        str  r2, [r1], #4
        cmp  r2, #0
        beq  ende
        b    loop

@ Ende des Programms
@ Stop durch Endlos-Schleife
ende:   nop
        b    ende

tab1:   .word 0x11111111, 0x22222222
        .word 0x55555555, 0x66666666
    
```

Schleife



Zur Programmierung
erforderliche Kenntnisse

- 1) Struktur der
Assemblerzeile
- 2) Programmiermodell des
Prozessors
- 3) Assembler-Befehle
- 4) Assembler-Direktiven
- 5) Adressierungsarten
- 6) Zahlendarstellung

Programmbeispiel

```
.syntax unified
.global asm2a
.text
asm2a:  adr  r0, tab1
        ldr  r1, =0x20000000
loop:   ldr  r2, [r0], #4
        str  r2, [r1], #4
        cmp  r2, #0
        beq  ende
        b    loop

@ Ende des Programms
@ Stop durch Endlos-Schleife
ende:   nop
        b    ende

tab1:   .word 0x11111111, 0x22222222
        .word 0x55555555, 0x66666666
```

Tabelle



Zur Programmierung
erforderliche Kenntnisse

- 1) Struktur der
Assemblerzeile
- 2) Programmiermodell des
Prozessors
- 3) Assembler-Befehle
- 4) Assembler-Direktiven
- 5) Adressierungsarten
- 6) Zahlendarstellung

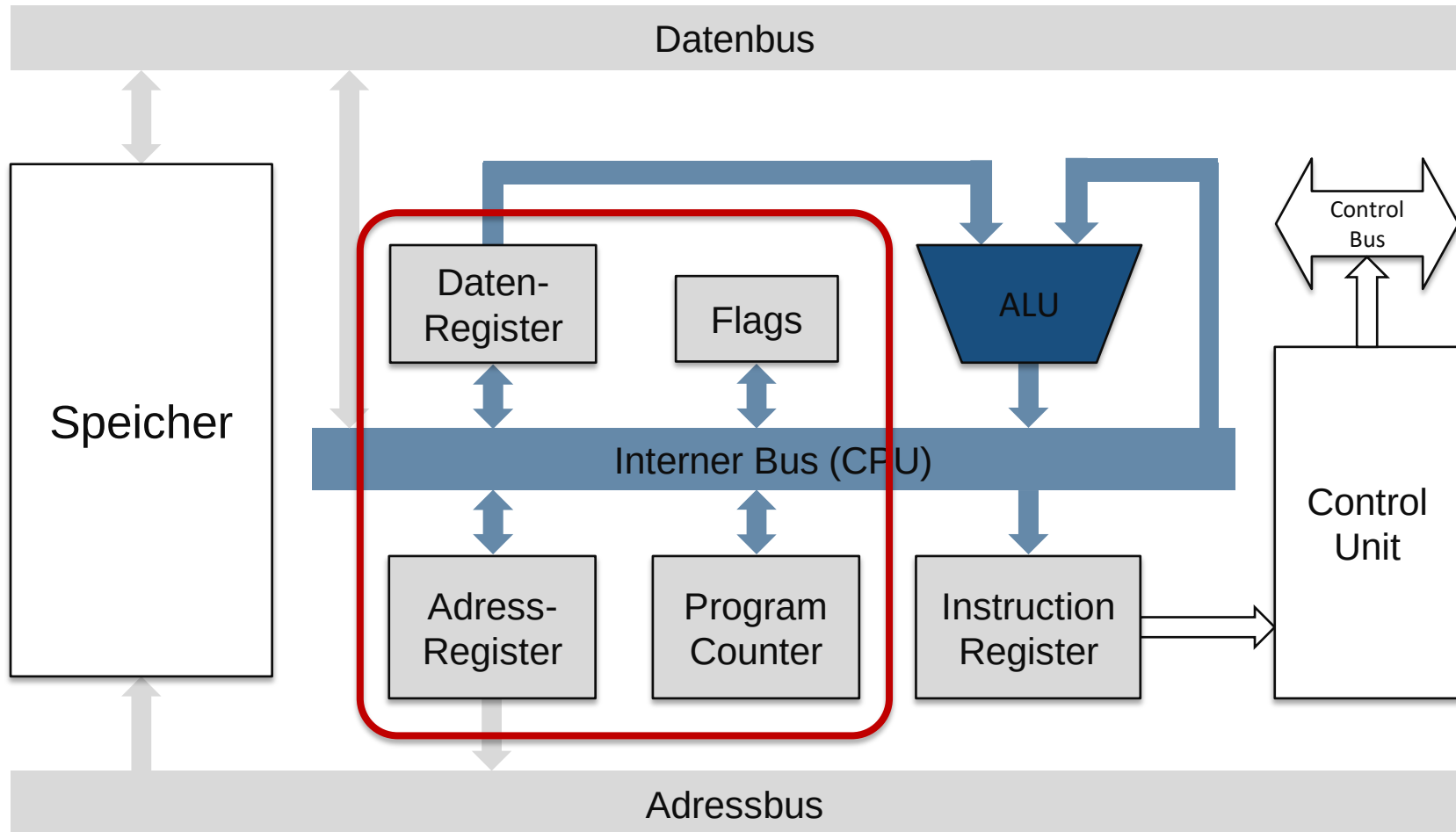
Assembler-Zeile

Labelfeld	Operationsfeld	Operandenfeld	Kommentarfeld
-----------	----------------	---------------	---------------

- Labelfeld:** leer
symbolischer Name (Konstante, Variable, Programmadresse)
Kommentarzeile
- Operationsfeld:** Anweisungen für den Prozessor (Assembler-Befehl)
Anweisungen für den Übersetzer (Assembler-Direktive)
- Operandenfeld:** enthält 0, 1 oder mehrere Operanden
(z.B. bei mov: 1. Operand → Zielregister, 2. Operand → absolute Zahl)
- Kommentarfeld:** in jeder Zeile, die eine Operation enthält, kann bis zum Zeilenende ein Kommentar eingetragen werden

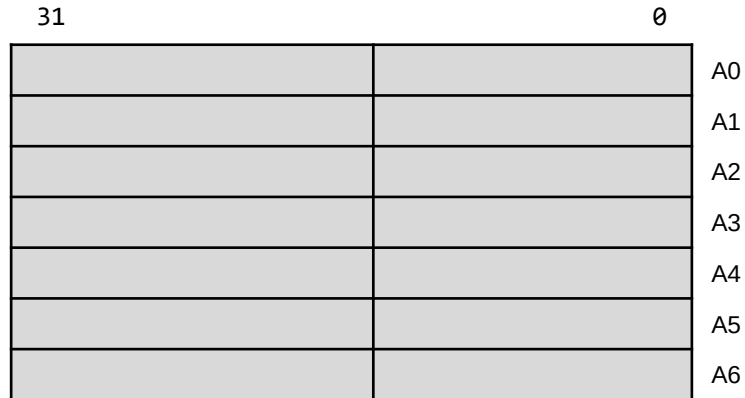
Registermodell

Prozessorkern – Überblick

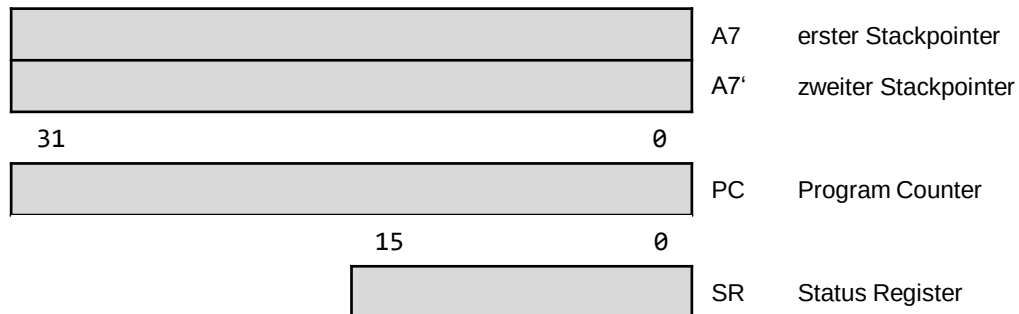
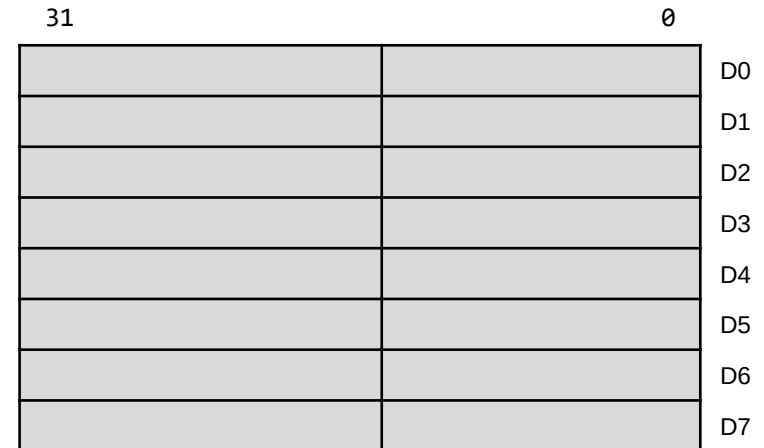


Registermodell MC68000

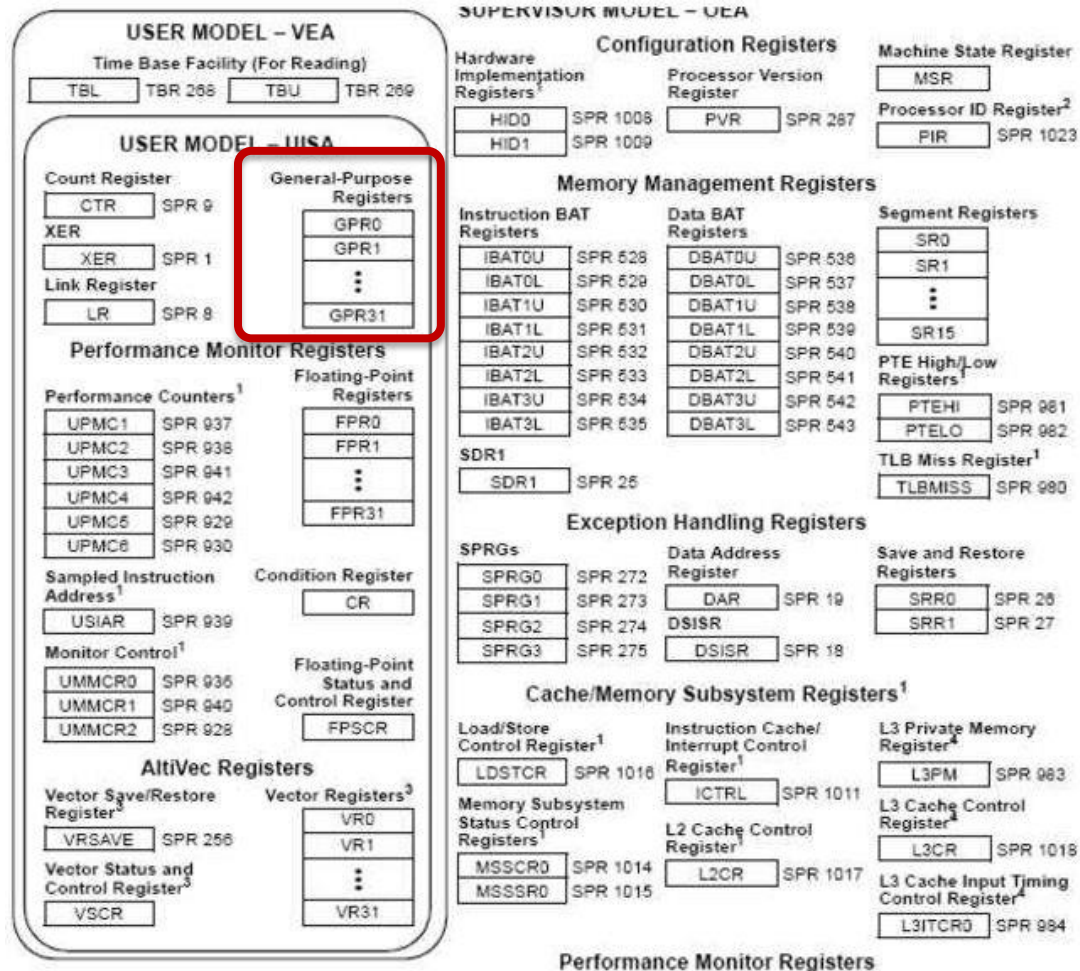
Sieben Adressregister



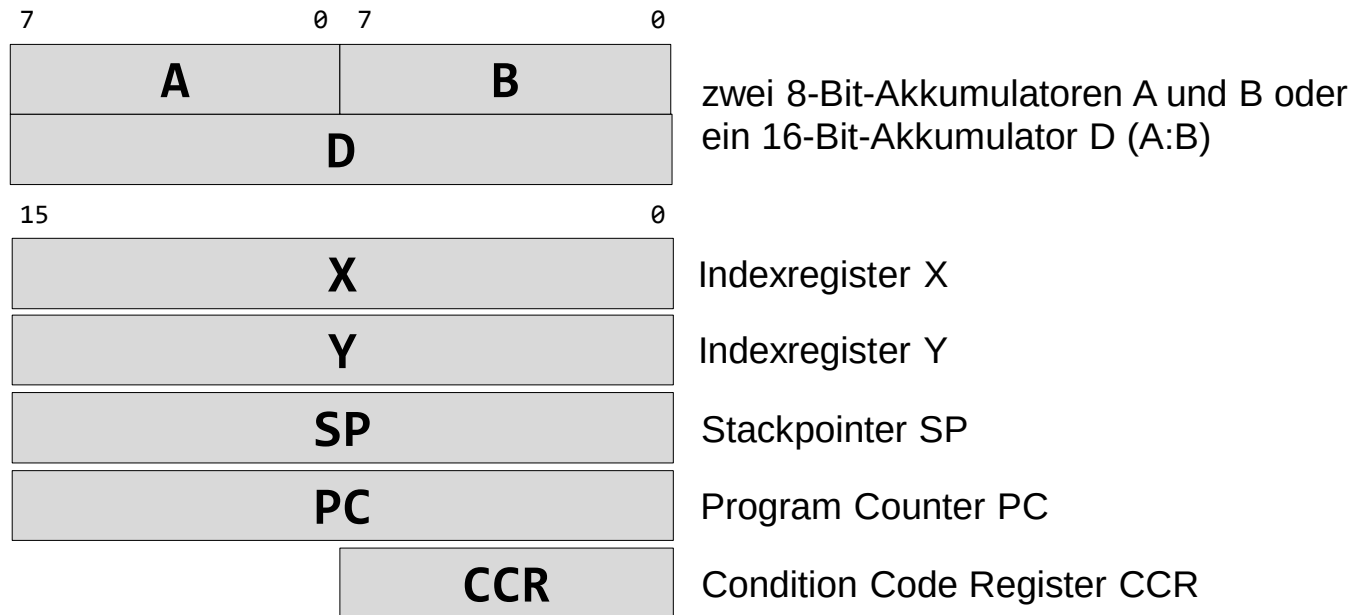
Acht Datenregister



Registermodell PowerPC



Registermodell 68HCS12



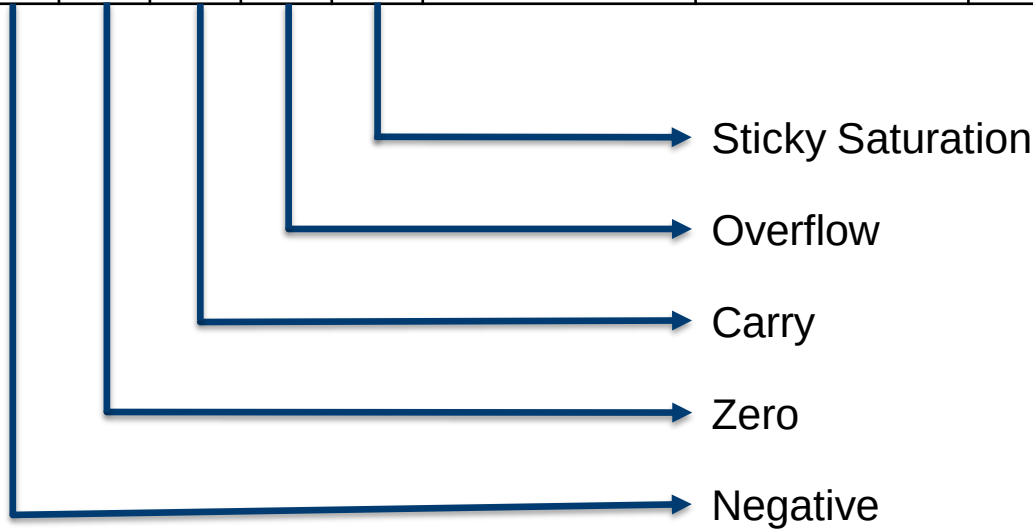
Registermodell *STM32L476RG*

31	R0	0	General-Purpose Registers
31	R1	0	
31	R2	0	
31	R3	0	
31	R4	0	
31	R5	0	
31	R6	0	
31	R7	0	
31	R8	0	
31	R9	0	
31	R10	0	
31	R11	0	
31	R12	0	
31	SP (R13)	0	Stack Pointer (SP)
31	LR (R14)	0	Link Register (LR)
31	PC (R15)	0	Program Counter (PC)
31	PSR	0	Program Status Register (PSR)

Registermodell *STM32L476RG*

Program Status Register (PSR)

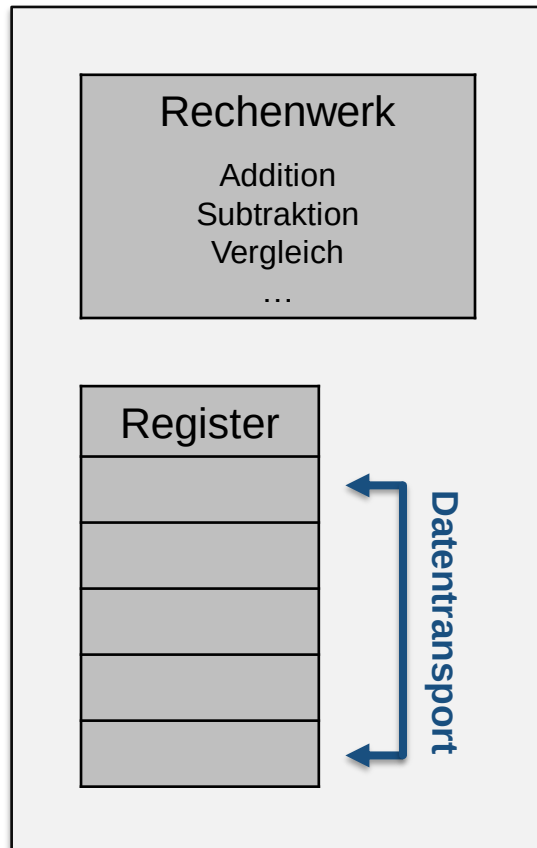
31	30	29	28	27	26	...	20	19	...	16	15	...	10	9	8	...	0
N	Z	C	V	Q	Reserved			GE[3;0]			ICI / IT			Res.		ISR_NUMBER	



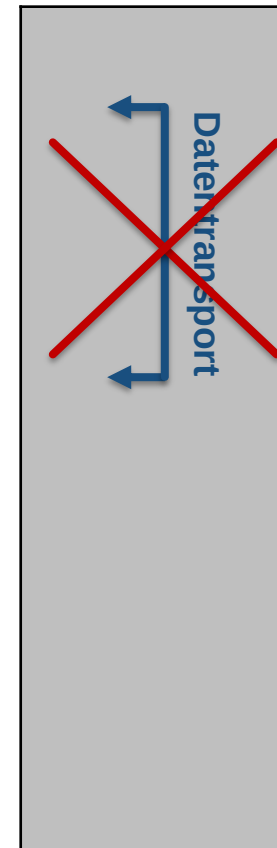
Daten bewegen

Datentransport

Prozessor

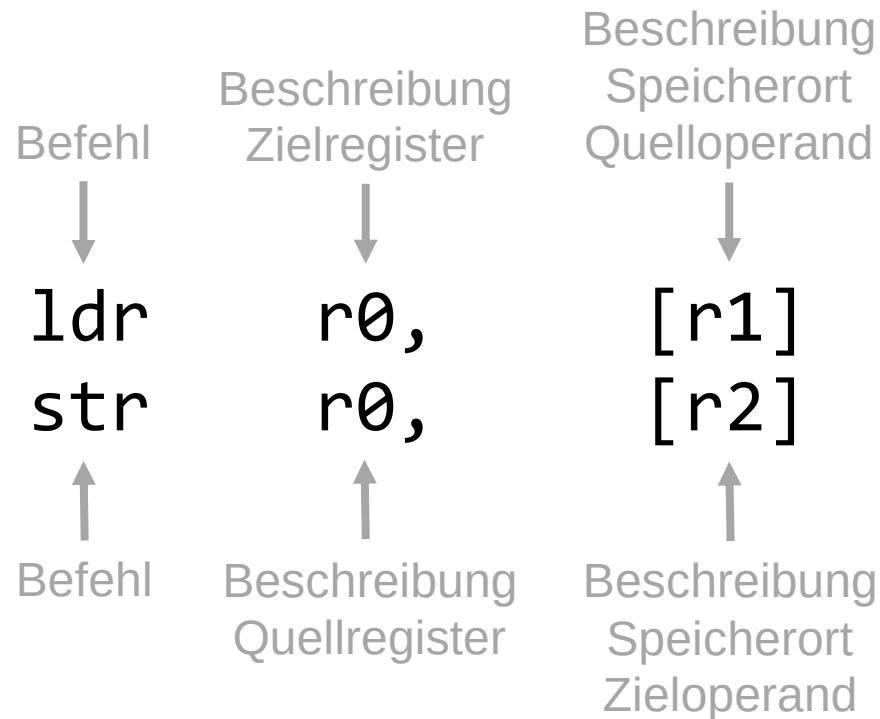


RAM



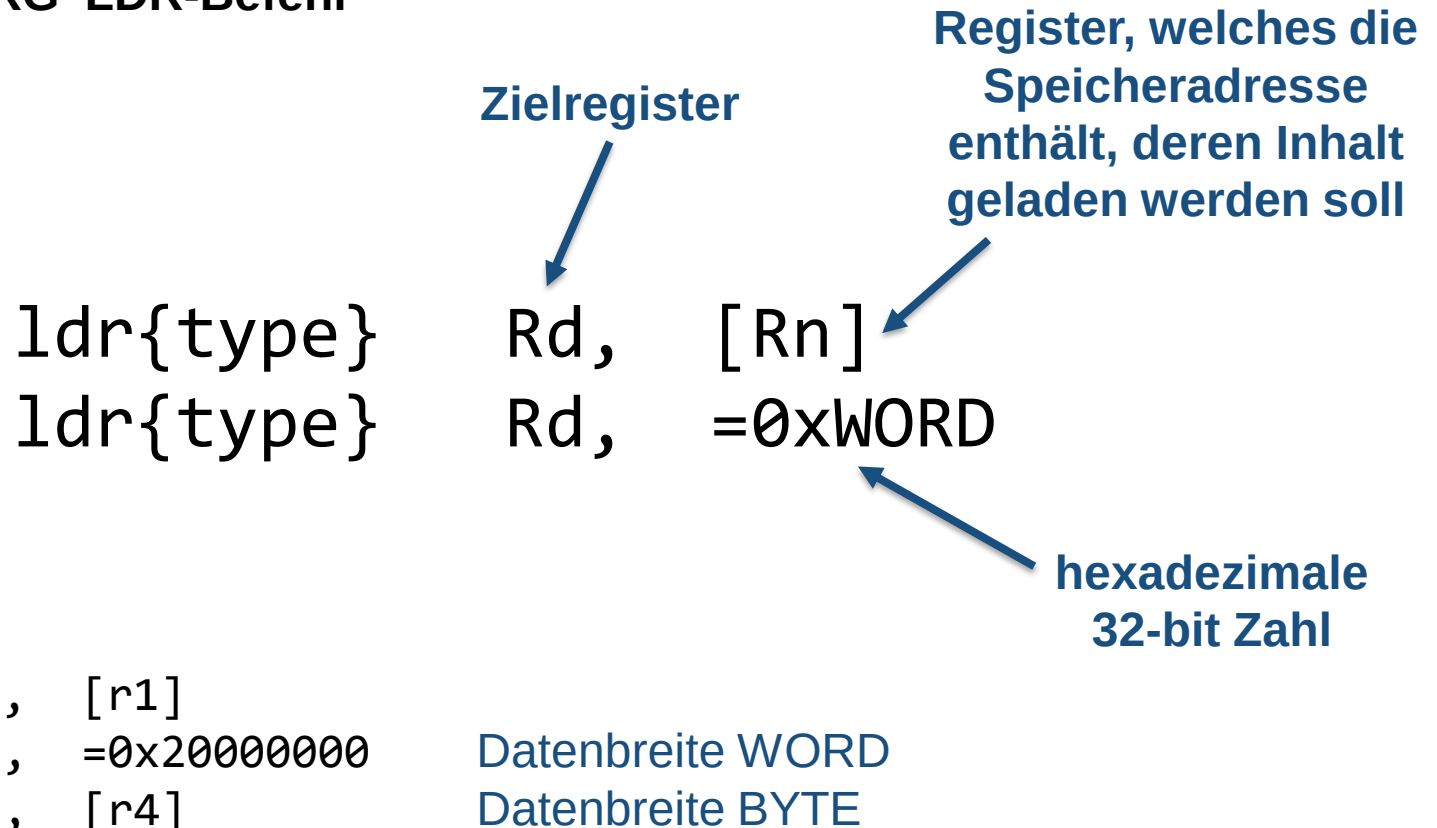
Befehlsgruppe *Daten bewegen*

Befehlsaufbau bei einer RISC-Architektur (Load / Store)



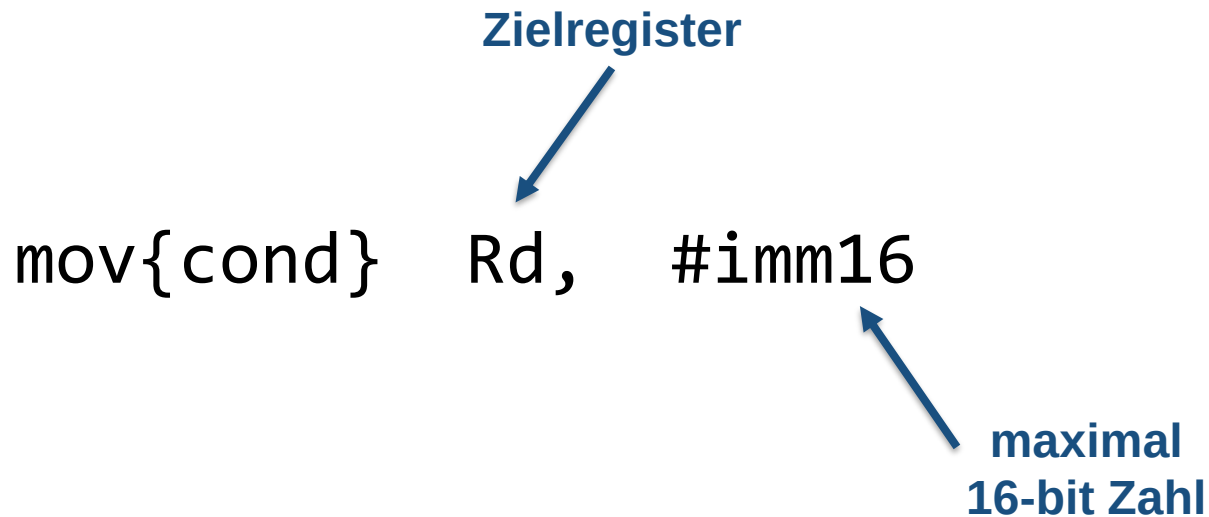
Befehlsgruppe *Daten bewegen*

STM32L476RG LDR-Befehl



Befehlsgruppe *Daten bewegen*

STM32L476RG MOV-Befehl

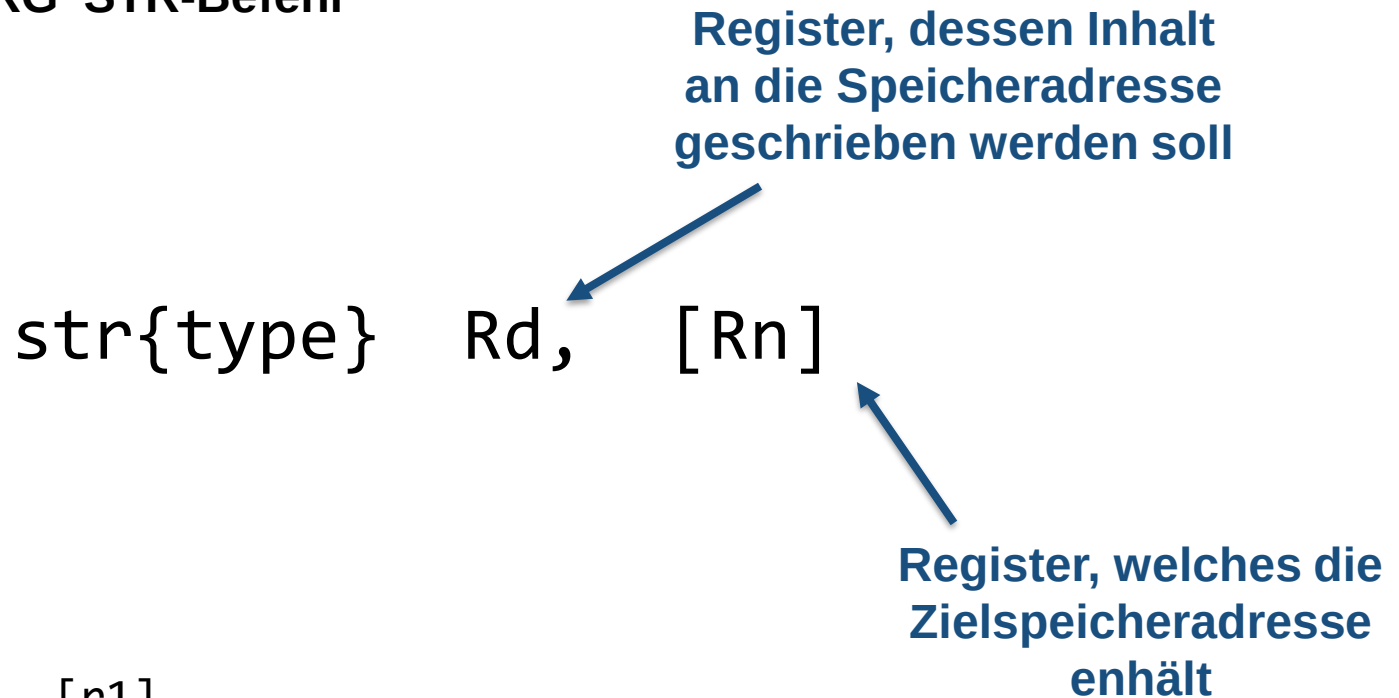


Beispiele:

```
ldr r0, #5
ldr r2, #0xF
```

Befehlsgruppe *Daten bewegen*

STM32L476RG STR-Befehl



Beispiel:

str r0, [r1]

Adressierungsarten?

Befehlsgruppe *Daten bewegen*

Programming Manual

LDR and STR, immediate offset

Load and Store with immediate offset, pre-indexed immediate offset, or post-indexed immediate offset.

Syntax

MOV and MVN

Move and Move NOT.

Syntax

MOV{S}{cond} Rd, Operand2

MOV{cond} Rd, #imm16

MVN{S}{cond} Rd, Operand2

Where:

- 'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation (see [Conditional execution on page 65](#)).
- 'cond' is an optional condition code (see [Conditional execution on page 65](#)).
- 'Rd' is the destination register.
- 'Operand2' is a flexible second operand (see [Flexible second operand on page 60](#)) for details of the options.
- 'imm16' is any value in the range 0—65535.

Operation

The MOV instruction copies the value of *operand2* into *Rd*.

When *operand2* in a MOV instruction is a register with a shift other than LSL #0, the preferred syntax is the corresponding shift instruction:

- ASR{S}{cond} Rd, Rm, #n is the preferred syntax for MOV{S}{cond} Rd, Rm, ASR #n
- LSL{S}{cond} Rd, Rm, #n is the preferred syntax for MOV{S}{cond} Rd, Rm, LSL #n if n != 0

...

offset

offset, two words
two words
two words

er)

ds

nly)

[execution on page 65](#))

based

address is the contents of *Rn*

-word operations

Befehlsgruppe *Daten bewegen*

LDR and STR, immediate offset

Load and Store with immediate offset, pre-indexed immediate offset, or post-indexed immediate offset.

Syntax

```
op{type}{cond} Rt, [Rn {, #offset}]; immediate offset
op{type}{cond} Rt, [Rn, #offset]!; pre-indexed
op{type}{cond} Rt, [Rn], #offset; post-indexed
opD{cond} Rt, Rt2, [Rn {, #offset}]; immediate offset, two words
opD{cond} Rt, Rt2, [Rn, #offset]!; pre-indexed, two words
opD{cond} Rt, Rt2, [Rn], #offset; post-indexed, two words
```

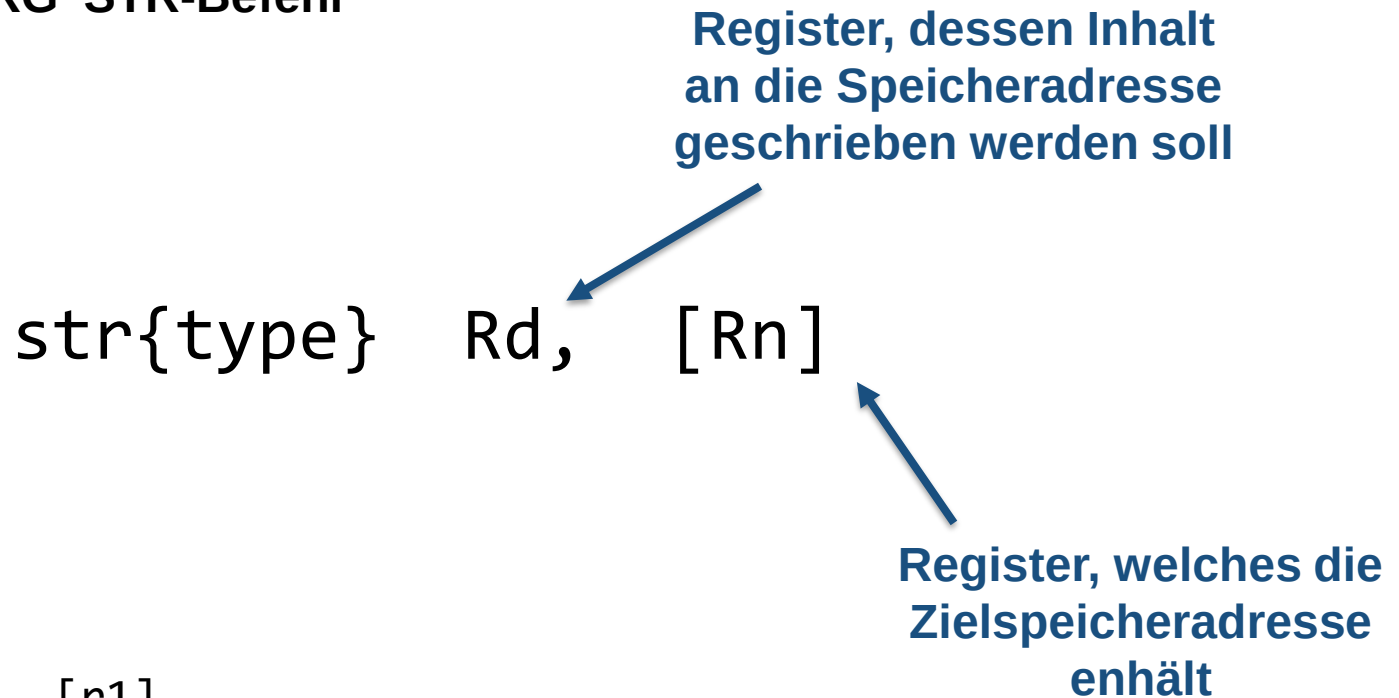
Where:

- 'op' is either LDR (load register) or STR (store register)
- 'type' is one of the following:
 - B: Unsigned byte, zero extends to 32 bits on loads
 - SB: Signed byte, sign extends to 32 bits (LDR only)
 - H: Unsigned halfword, zero extends to 32 bits on loads
 - SH: Signed halfword, sign extends to 32 bits (LDR only)
 - : Omit, for word
- 'cond' is an optional condition code (see [Conditional execution on page 65](#))
- 'Rt' is the register to load or store
- 'Rn' is the register on which the memory address is based
- 'offset' is an offset from Rn. If offset is omitted, the address is the contents of Rn
- 'Rt2' is the additional register to load or store for two-word operations

...

Befehlsgruppe *Daten bewegen*

STM32L476RG STR-Befehl



Beispiel:

str r0, [r1]

Adressierungsarten?

Adressierungsarten

R0	0x00000010
R1	
R2	
R3	
R4	0x0000000F
R5	
R6	
R7	
R8	0x20000000
R9	
R10	
R11	
R12	
SP	
LR	
PC	

immediate (unmittelbar)

mov r0, #16

mov r4, #0xF

ldr r8, =0x20000000

Adressierungsarten

R0	0x00000010
R1	
R2	
R3	
R4	0x0000000F
R5	
R6	
R7	
R8	0x20000000
R9	
R10	
R11	
R12	
SP	
LR	
PC	

immediate (unmittelbar)

mov r0, #16

immediate
decimal

mov r4, #0xF

immediate
hexadecimal

ldr r8, =0x20000000

pseudo-instruction

Adressierungsarten

R0	0x20000010
R1	
R2	
R3	
R4	
R5	
R6	
R7	0x00000042
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Register, indirect

ldr r7, [r0]

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	0x00000042	...	...	...
0x20000020	...	...	...	...
...	...	...	...	...

Tabellen kopieren: Pointer für jedes Element neu initialisieren?

Adressierungsarten

Register, immediate offset

ldr r7, [r0, #4]

$0x200000010 + 4$

R0	0x200000010
R1	
R2	
R3	
R4	
R5	
R6	
R7	0x00000042
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x200000000	...	...	...	...
0x200000010	...	0x00000042	...	...
0x200000020	...	...	...	...
...	...	...	...	...

Adressierungsarten

R0	0x20000010
R1	0x0000000C
R2	
R3	
R4	
R5	
R6	
R7	0x00000042
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Register, register offset

ldr r7, [r0, r1]

0x20000010
+ 0x0000000C

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	...	...	0x00000042
0x20000020	...	...	...	...
...	...	...	...	...

Adressierungsarten

R0	0x20000010
R1	
R2	
R3	
R4	
R5	
R6	
R7	0x00000042
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Pre-Increment

ldr r7, [r0, #4]!

0x20000010
+
4

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	0x00000042	...	...
0x20000020	...	...	...	...
...	...	...	...	...

Adressierungsarten

R0	0x20000018
R1	
R2	
R3	
R4	
R5	
R6	
R7	0x00000042
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Pre-Decrement

ldr r7, [r0, #-4]!

0x20000018
-
4

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	0x00000042	...	...
0x20000020	...	...	...	...
...	...	...	...	...

Adressierungsarten

R0	0x20000010
R1	
R2	
R3	
R4	
R5	
R6	
R7	0x00000042
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Post-Increment

ldr r7, [r0], #4

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	0x00000042	...	...	...
0x20000020	...	...	...	...
...	...	...	...	...

Adressierungsarten

R0	0x20000014
R1	
R2	
R3	
R4	
R5	
R6	
R7	0x00000042
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Post-Increment

ldr r7, [r0], #4

0x20000010
+ 4

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	0x00000042	...	...	...
0x20000020	...	...	...	...
...	...	...	...	...

Adressierungsarten

R0	0x20000010
R1	
R2	
R3	
R4	
R5	
R6	
R7	0x00000042
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Post-Decrement

ldr r7, [r0], #-4

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	0x00000042	...	...	...
0x20000020	...	...	...	...
...	...	...	...	...

Beispiel: Kopierschleife

1. Daten kopieren
2. Pointer verändern

R0	0x20000000
R1	0x20000100
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b  loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100				
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

1. Daten kopieren

2. Pointer verändern

R0	0x20000000
R1	0x20000100
R2	0x11111111
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100				
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

1. Daten kopieren

2. Pointer verändern

R0	0x20000004
R1	0x20000100
R2	0x11111111
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100				
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

R0	0x20000004
R1	0x20000100
R2	0x11111111
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b    loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100	0x11111111			
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

R0	0x20000004
R1	0x20000104
R2	0x11111111
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100	0x11111111			
0x20000110				...
...	...	...	...	...

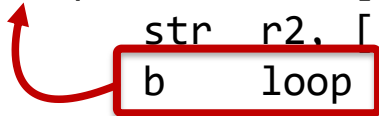
Beispiel: Kopierschleife

R0	0x20000004
R1	0x20000104
R2	0x11111111
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b  loop
...

```



Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100	0x11111111			
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

1. Daten kopieren

2. Pointer verändern

R0	0x20000004
R1	0x20000104
R2	0x22222222
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100	0x11111111			
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

1. Daten kopieren

2. **Pointer verändern**

R0	0x20000008
R1	0x20000104
R2	0x22222222
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100	0x11111111			
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

R0	0x20000008
R1	0x20000104
R2	0x22222222
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b    loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100	0x11111111	0x22222222		
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

R0	0x20000008
R1	0x20000108
R2	0x22222222
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr r0, =0x20000000
ldr r1, =0x20000100
loop: ldr r2, [r0], #4
      str r2, [r1], #4
      b loop
...

```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100	0x11111111	0x22222222		
0x20000110				...
...	...	...	...	...

Beispiel: Kopierschleife

R0	...
R1	...
R2	...
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```

...
ldr  r0, =0x20000000
ldr  r1, =0x20000100
loop: ldr  r2, [r0], #4
      str  r2, [r1], #4
      b    loop
...

```

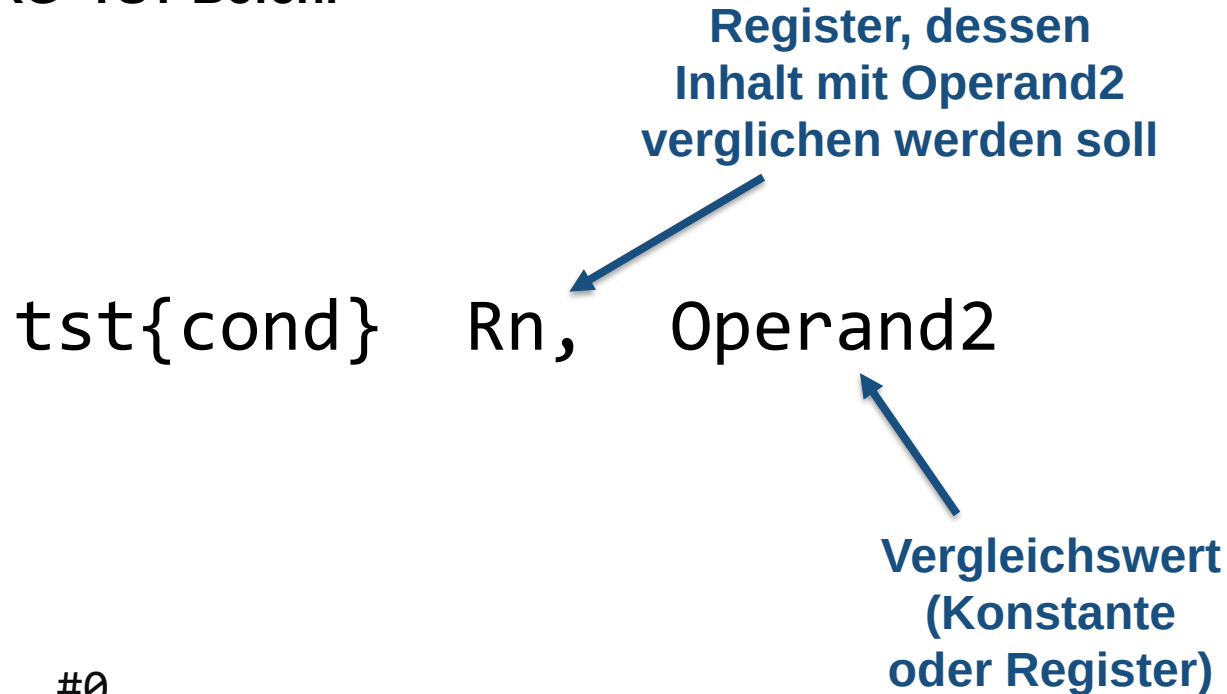
und so weiter...

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11111111	0x22222222	0x33333333	0x44444444
0x20000010	0x55555555	0x66666666	0x77777777	0x88888888
0x20000020	0x99999999	...	...	...
...	...	...	...	...
0x20000100	0x11111111	0x22222222	0x33333333	0x44444444
0x20000110	0x55555555	0x66666666	0x77777777	...
...	...	...	...	...

Daten vergleichen und Sprungbefehle

Befehlsgruppe *Daten vergleichen*

STM32L476RG TST-Befehl



Beispiel:

```
tst  r0,  #0
tst  r1,  r2
```

Befehlsgruppe *Daten vergleichen*

TST and TEQ

Test bits and Test Equivalence.

Syntax

TST{cond} Rn, Operand2

TEQ{cond} Rn, Operand2

Where:

- 'cond' is an optional condition code (see *Conditional execution on page 65*).
- 'Rn' is the register holding the first operand.
- 'Operand2' is a flexible second operand (see *Flexible second operand on page 60*) for details of the options.

Operation

These instructions test the value in a register against *operand2*. They update the condition flags based on the result, but do not write the result to a register.

The TST instruction performs a bitwise AND operation on the value in *Rn* and the value of *operand2*. This is the same as the ANDS instruction, except that it discards the result.

To test whether a bit of *Rn* is 0 or 1, use the TST instruction with an *operand2* constant that has that bit set to 1 and all other bits cleared to 0.

The TEQ instruction performs a bitwise exclusive OR operation on the value in *Rn* and the value of *operand2*. This is the same as the EORS instruction, except that it discards the result.

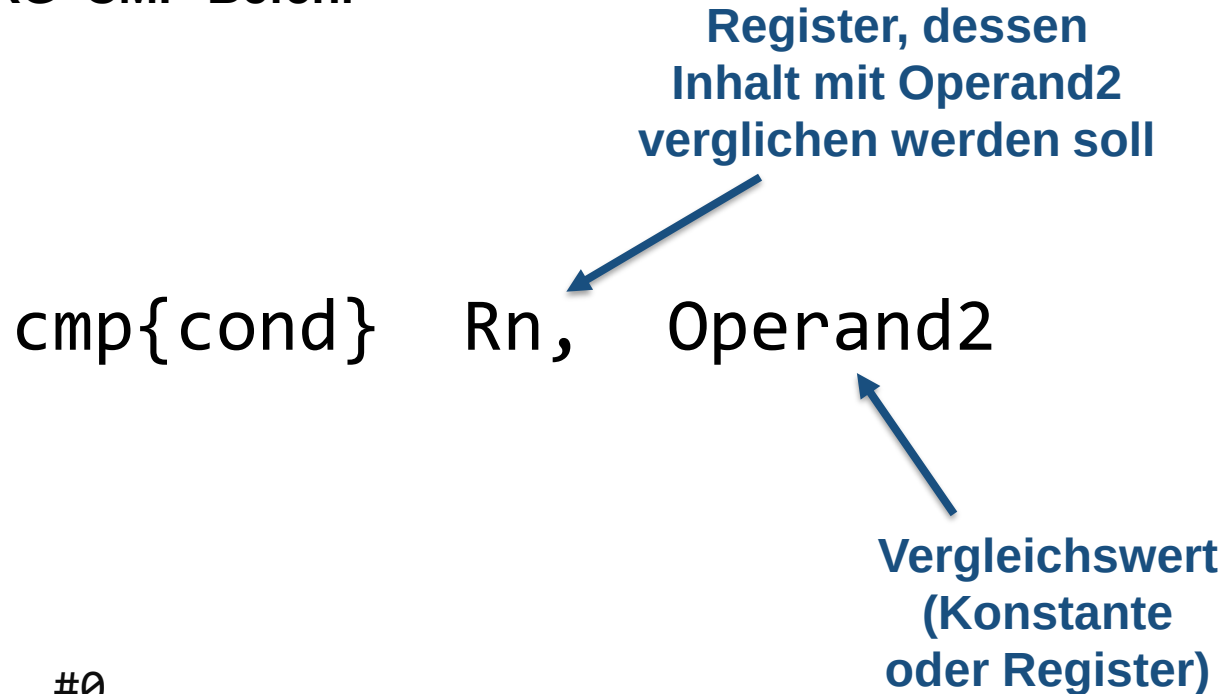
Use the TEQ instruction to test if two values are equal without affecting the V or C flags.

TEQ is also useful for testing the sign of a value. After the comparison, the N flag is the logical exclusive OR of the sign bits of the two operands.

...

Befehlsgruppe *Daten vergleichen*

STM32L476RG CMP-Befehl



Beispiel:

```
cmp r0, #0
cmp r1, r2
```

Befehlsgruppe *Daten vergleichen*

CMP and CMN

Compare and Compare Negative.

Syntax

`CMP{cond} Rn, Operand2`

`CMN{cond} Rn, Operand2`

Where:

- 'cond' is an optional condition code (see [Conditional execution on page 65](#)).
- 'Rn' is the register holding the first operand.
- 'Operand2' is a flexible second operand (see [Flexible second operand on page 60](#)) for details of the options.

Operation

These instructions compare the value in a register with *operand2*. They update the condition flags on the result, but do not write the result to a register.

The CMP instruction subtracts the value of *operand2* from the value in *Rn*. This is the same as a SUBS instruction, except that the result is discarded.

The CMN instruction adds the value of *operand2* to the value in *Rn*. This is the same as an ADDS instruction, except that the result is discarded.

Restrictions

In these instructions:

- Do not use PC.
- *Operand2* must not be SP.

Condition flags

These instructions update the N, Z, C and V flags according to the result.

...

Befehlsgruppe *Daten vergleichen*

CMP and CMN

Compare and Compare Negative.

Syntax

CMP{cond} Rn, Operand2

CMN{cond} Rn, Operand2

Where:

- 'cond' is an optional condition code (see [Conditional execution on page 65](#)).
- 'Rn' is the register.
- 'Operand2' is the second operand. For details of the syntax, see the ARM DDI.

Operation

These instructions compare the value in Rn with the value in Operand2. They set the condition flags on the result, but do not write the result to the register file.

The CMP instruction subtracts the value in Operand2 from the value in Rn. This is the same as a SUBS instruction, except that the result is discarded.

The CMN instruction adds the value in Operand2 to the value in Rn. This is the same as an ADDS instruction, except that the result is discarded.

Restrictions

In these instructions:

- Do not use PC.
- Operand2 must not be SP.

Condition flags

These instructions update the N, Z, C and V flags according to the result.

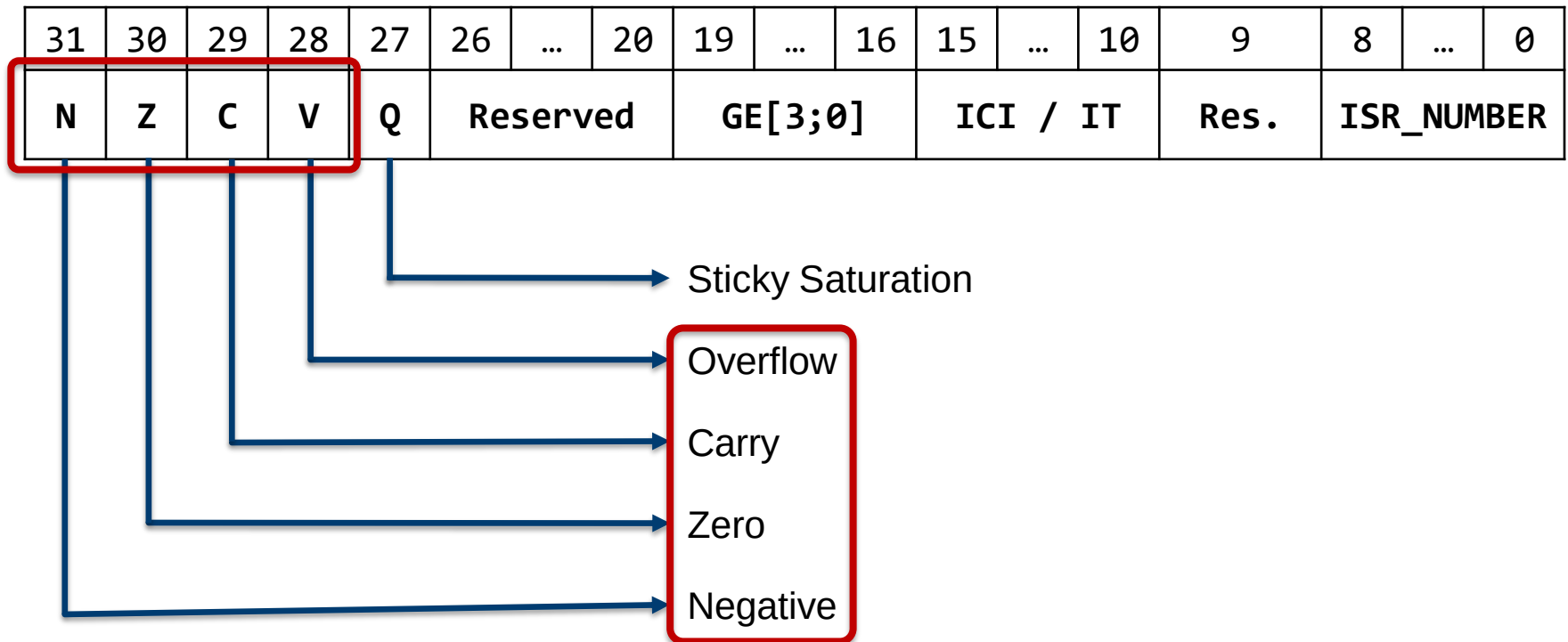
...

Condition flags

These instructions update the N, Z, C and V flags according to the result.

Registermodell *STM32L476RG*

Program Status Register (PSR)



Befehlsgruppe *Sprungbefehle*

STM32L476RG Branch-Befehl

Condition:
Bedingung, bei der
ein Sprung erfolgt

b{cond} label
b1{cond} label

**Auswertung der
PSR-Flags**

**Label im Programm, zu
welchem gesprungen
werden soll**

Beispiel:

```
beq    loop
b      ende
```


Befehlsgruppe *Sprungbefehle*

Branch-Conditions

beq	label	branch if equal
bne	label	branch if not equal
bgt	label	branch if greater than
bge	label	branch if greater or equal
blt	label	branch if less than
ble	label	branch if less or equal
bvs	label	branch if overflow set
bvc	label	branch if overflow clear
bcs	label	branch if carry set
bcc	label	branch if carry clear

Befehlsgruppe *Sprungbefehle*

Suffix	Flags	Meaning
EQ	Z = 1	Equal
NE	Z = 0	Not equal
CS or HS	C = 1	Higher or same, unsigned $\geq$
CC or LO	C = 0	Lower, unsigned $<$
MI	N = 1	Negative
PL	N = 0	Positive or zero
VS	V = 1	Overflow
VC	V = 0	No overflow
HI	C = 1 and Z = 0	Higher, unsigned $>$
LS	C = 0 or Z = 1	Lower or same, unsigned $\leq$
GE	N = V	Greater than or equal, signed $\geq$
LT	N $\neq$ V	Less than, signed $<$
GT	Z = 0 and N = V	Greater than, signed $>$
LE	Z = 1 and N $\neq$ V	Less than or equal, signed $\leq$
AL	Can have any value	Always. This is the default when no suffix is specified.

...

Befehlsgruppe *Sprungbefehle*

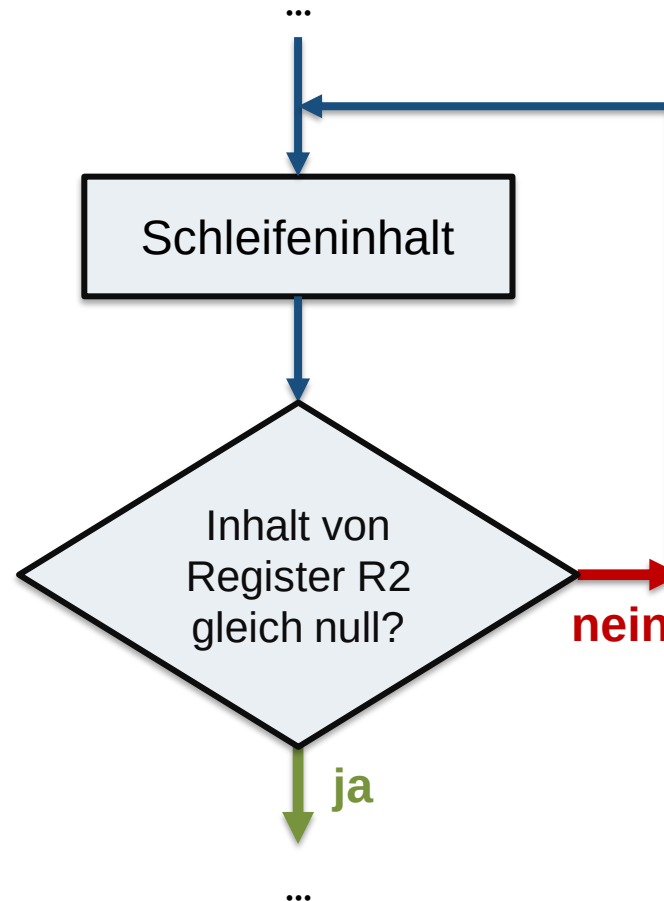
Schleife

```

...
loop:  ldr  r2, [r0], #4
        str  r2, [r1], #4
        cmp  r2, #0
        beq  ende
...

```

Diagramm zur Schleife: Ein roter Pfeil führt von der Zeile `beq ende` zurück zum Label `loop`. Ein grüner Pfeil zeigt nach unten.



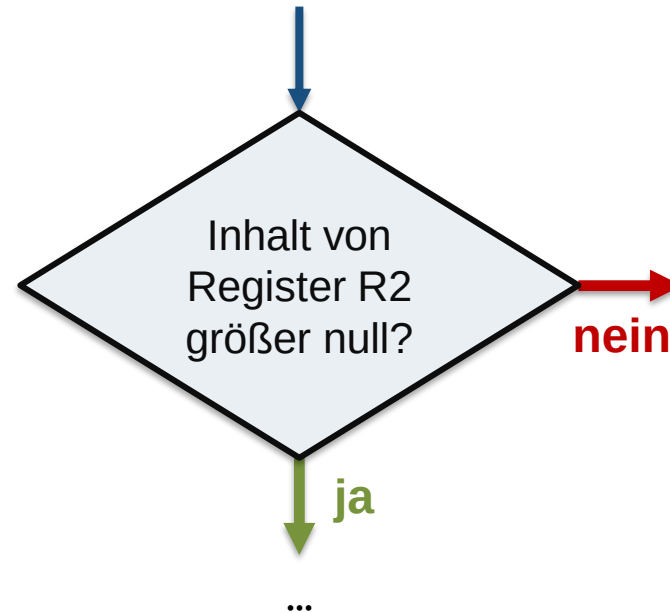
Befehlsgruppe *Sprungbefehle*

Verzweigung

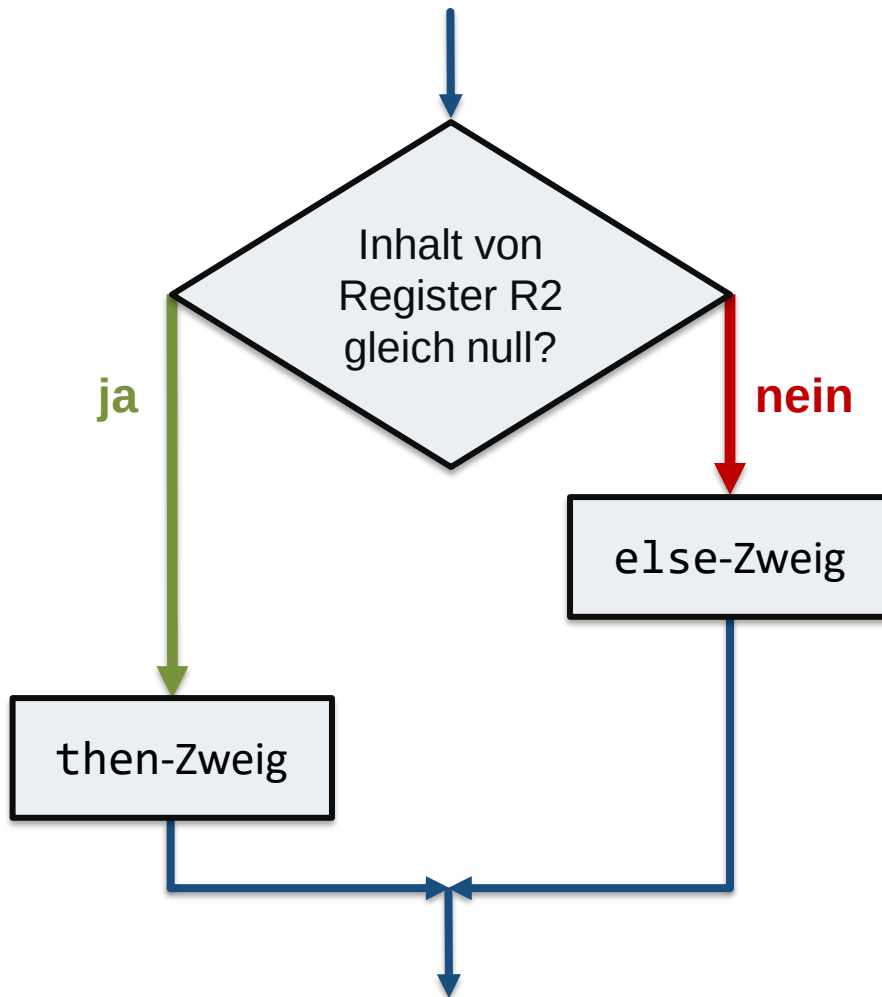
```

...
loop:  ldr  r2, [r0], #4
        str  r2, [r1], #4
        cmp  r2, #0
        bgt  ziel1
...
ziel1: ...
    
```

A green curved arrow points from the `bgt` instruction to the `ziel1` label. A red arrow points down from the `ziel1` label.



Befehlsgruppe *Sprungbefehle*



```

...
cmp    r2, #0
beq    then

```

```

...
b      all

```

```

then:  ...

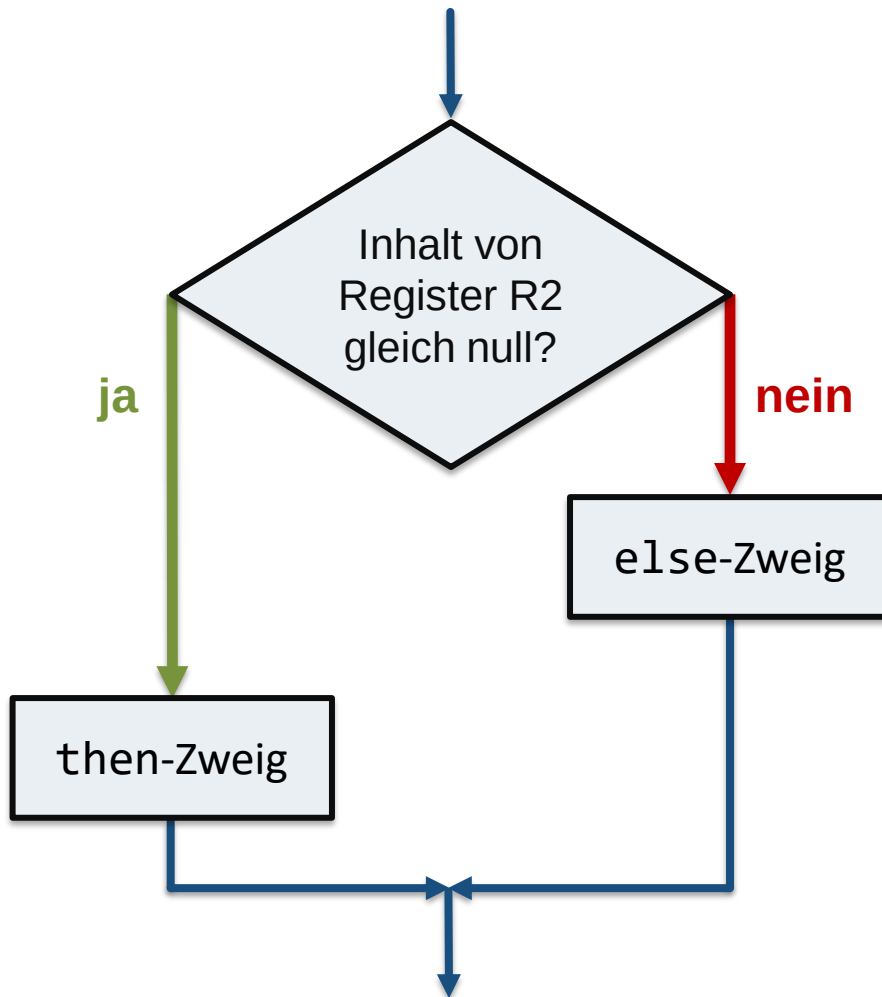
```

```

all:   ...

```

Befehlsgruppe *Sprungbefehle*



...

```
cmp    r2, #0
beq    then
```

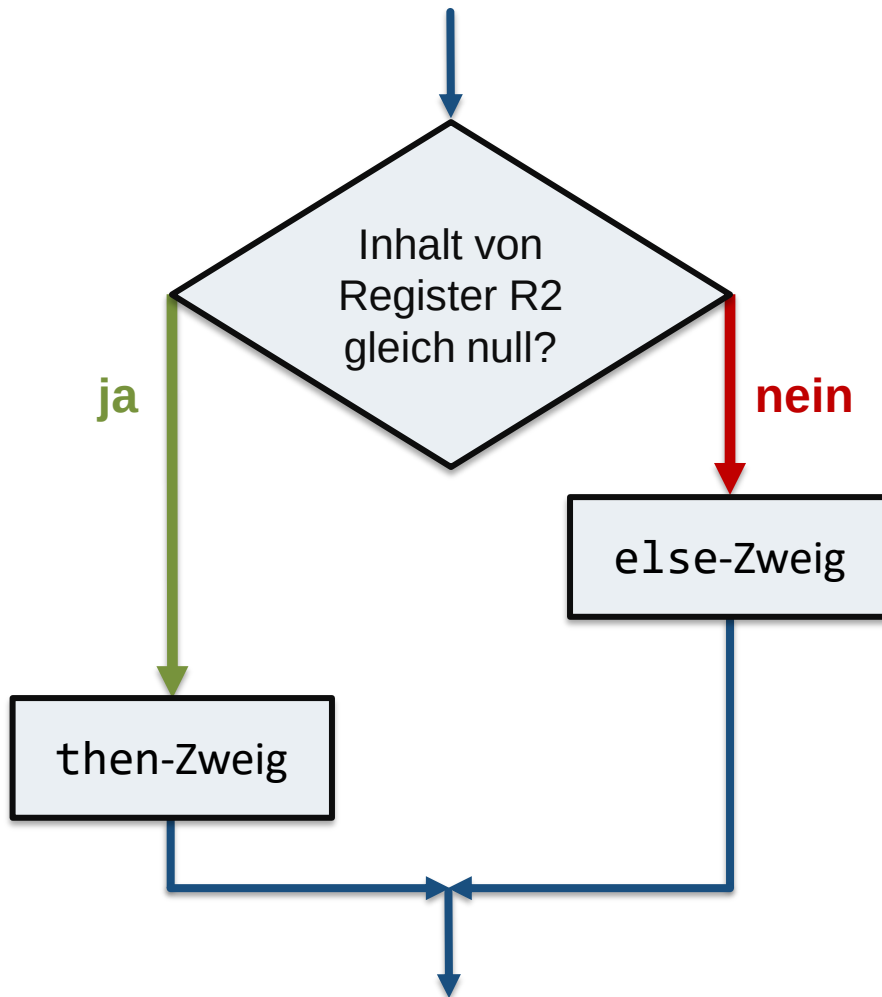
...

```
b      all
```

then: ...

all: ...

Befehlsgruppe *Sprungbefehle*



```

...
cmp    r2, #0
beq    then
...
b      all
then:  ...
all:   ...
  
```